



CitaBella

Sistema Integral de Gestión para Peluquerías

Autor: Ivan López Molina

Ciclo: 2º DAM

Curso: 2025 - 2026

Centro: IES La Vereda

Tutor: Víctor Pla Soler



*A Ruth Molina, por prestarme su negocio como caso real
y tomarse el tiempo de explicarme cómo funciona una
peluquería desde dentro.*



ABSTRACT / RESUMEN / RESUM

Abstract

CitaBella is a comprehensive management system for hair salons developed as the final project of the Multiplatform Application Development (DAM) program. It fully digitizes the daily operations of a real salon appointment scheduling, client and employee management, service catalog, and product inventory replacing manual paper agendas and WhatsApp communications with a modern, secure, containerized solution.

The system is built on a client-server architecture fully orchestrated with Docker Compose. The backend is a REST API developed in Java 17 with Spring Boot 3 and secured by stateless JWT authentication, the frontend is a Single Page Application built with Angular 21, and the relational model is managed by MySQL 8. A native Android application, developed with Flutter and an advanced WebView, embeds the web interface to deliver a complete mobile experience without duplicating business logic.

At the time of this submission, the Minimum Viable Product (MVP) is 100% complete, tested, and deployable with a single command. It delivers the full appointment lifecycle (state machine with overlap detection), role-based access control (Admin, Employee, Client), a public facing product and service catalog, and thorough technical documentation including an interactive OpenAPI explorer. The solution constitutes a fully functional academic prototype meeting all technical standards defined for the MVP, with sales management, advanced inventory and automated notifications planned for future development phases.

Resumen

CitaBella es un sistema integral de gestión para peluquerías desarrollado como proyecto final del ciclo de Desarrollo de Aplicaciones Multiplataforma (DAM). El sistema digitaliza completamente la operativa diaria de una peluquería real, incluyendo la gestión de citas, clientes, empleados, catálogo de servicios e inventario de productos. De este modo, sustituye la agenda de papel y la mensajería instantánea por una solución moderna, segura y contenerizada.

La aplicación sigue una arquitectura cliente-servidor orquestada con Docker Compose. El backend consiste en una API REST desarrollada con Java 17 y Spring Boot 3, protegida mediante autenticación JWT sin estado. El frontend es una SPA construida con Angular 21, mientras que el sistema de base de datos utiliza MySQL 8. Además, se ha desarrollado una aplicación nativa para Android con Flutter y un WebView avanzado, que integra la interfaz web para ofrecer una experiencia móvil completa sin duplicar la lógica de negocio.

En el momento de esta entrega, el Producto Mínimo Viable (MVP) está completamente finalizado, probado y desplegable con un solo comando. El sistema cubre el ciclo de vida completo de una cita, incluyendo una máquina de estados y detección de solapamientos. También incorpora control de acceso por roles (Administrador, Empleado y Cliente), una zona pública con catálogo de servicios y productos, y documentación técnica completa con un explorador interactivo de la API.

El resultado es un prototipo académico funcional que cumple los estándares técnicos definidos para el MVP entregado.

Resum

CitaBella és un sistema integral de gestió per a perruqueries desenvolupat com a projecte final del cicle de Desenvolupament d'Aplicacions Multiplataforma (DAM). El sistema digitalitza completament l'operativa diària d'una perruqueria real, incloent-hi la gestió de cites, clients, empleats, catàleg de serveis i inventari de productes. D'aquesta manera, substitueix l'agenda de paper i la missatgeria instantània per una solució moderna, segura i contenidoritzada.

L'aplicació segueix una arquitectura client-servidor orquestrada amb Docker Compose. El backend consisteix en una API REST desenvolupada amb Java 17 i Spring Boot 3, protegida mitjançant autenticació JWT sense estat. El frontend és una SPA creada amb Angular 21, mentre que el sistema de base de dades utilitza MySQL 8. A més, s'ha desenvolupat una aplicació mòbil per a Android mitjançant Flutter que incorpora un WebView avançat, que integra la interfície web per oferir una experiència mòbil completa sense duplicar la lògica de negoci.

En el moment d'aquest lliurament, el Producte Mínim Viable (MVP) està completament finalitzat, provat i desplegable amb una sola ordre. El sistema cobreix el cicle de vida complet d'una cita, incloent-hi una màquina d'estats i detecció de solapaments. També incorpora control d'accés per rols (Administrador, Empleat i Client), una zona pública amb catàleg de serveis i productes, i documentació tècnica completa amb un explorador interactiu de l'API.

ÍNDICE

1. INTRODUCCIÓN	7
1.1 Módulos implicados.....	7
1.2 Breve descripción del proyecto.....	7
1.2.1 Objetivo general.....	8
1.2.2 Objetivos específicos	8
1.2.3 Uso del proyecto	9
1.2.4 Tecnologías investigadas no vistas en clase.....	9
2. ESTUDIO PREVIO.....	10
2.1 Análisis del caso real.....	10
2.2 Necesidades detectadas	10
2.3 Estudio de soluciones existentes	10
2.4 Propuesta de valor y público objetivo.....	11
2.5 Análisis DAFO	12
2.6 Estrategia derivada del DAFO	12
3. PLAN DE TRABAJO	12
3.1 Metodología.....	12
3.2 Temporalización.....	13
3.3 Herramientas de seguimiento	14
4. DISEÑO.....	16
4.1 Diseño general (análisis).....	16
4.1.1 Diagrama de contexto	16
4.1.2 Historias de usuario	16
4.1.3 Flujos de usuario.....	17
4.1.4 Modelo conceptual	18
4.2 Diseño detallado (diseño técnico).....	19
4.2.1 Diagrama Entidad-Relación de la Base de Datos.....	19
4.2.2 Diagrama de clases (Backend).....	20
4.2.3 Arquitectura del sistema	21
4.2.4 Diseño de la API REST	21
4.2.5 Vistas implementadas del frontend	22

4.2.6 Navegación de la App Android (Flutter)	26
4.2.7 Esquema Docker	28
4.2.8 Justificación de decisiones técnicas	29
5. IMPLANTACIÓN	31
5.1 Backend — Spring Boot	31
5.1.1 Configuración del entorno	31
5.1.2 Docker Compose: Servicios levantados	31
5.1.3 Spring Boot: Estructura, paquetes, endpoints CRUD	32
5.1.4 Seguridad: JWT, roles	32
5.1.5 Validaciones de negocio	34
5.1.6 Scripts SQL e inicialización	34
5.1.7 Logs y auditoría	35
5.2 Frontend — Angular 21	35
5.2.1 Estructura y arquitectura de módulos	35
5.2.2 Guardas e interceptor JWT	36
5.2.3 Calendario de citas con FullCalendar	36
5.2.4 Sistema de temas oscuro/claro con Angular Signals	36
5.3 Aplicación móvil — Flutter	38
5.3.1 Arquitectura del contenedor WebView	38
5.3.2 Gestión de conectividad y estados del WebView	38
5.3.3 Funcionalidades nativas	38
5.4 DevOps y despliegue con Docker	39
5.4.1 Dockerfiles multietapa	39
5.4.2 Variables de entorno y persistencia	39
5.4.3 Comandos de operación	40
5.5 Gestión del código con Git y GitHub	40
5.5.1 Estrategia de ramas	40
5.5.2 Convención de commits	40
5.5.3 GitHub Projects y seguimiento de tareas	41
5.6 Pruebas del sistema	41
5.6.1 Pruebas de la API REST	41

5.6.2 Pruebas de integración manual	41
5.6.3 Pruebas de la aplicación móvil	42
5.6.4 Trabajo futuro en pruebas	42
6. RECURSOS	42
6.1 Hardware utilizado	42
6.2 Software y herramientas.....	42
6.3 Sistemas operativos	43
6.4 Repositorio y servicios externos	43
7. CONCLUSIONES	43
7.1 Grado de consecución de objetivos	43
7.2 Problemas encontrados y soluciones.....	44
7.3 Lecciones aprendidas	45
7.4 Mejoras futuras y roadmap	45
7.5 Cumplimiento de Resultados de Aprendizaje	46
8. REFERENCIAS Y BIBLIOGRAFÍA	48
9. ANEXOS.....	49
Anexo A — Entrevista con la propietaria del negocio	49
Anexo B — Reglas de negocio del sistema	51
Anexo C — Índice de imágenes	52
Anexo D — Historias de usuario.....	54
Anexo E — Pruebas de la API	55
Anexo F — Prevención de riesgos laborales	56
Anexo G — Instalación y despliegue del sistema.....	60
Anexo H — Estructura de documentación técnica	60

1. INTRODUCCIÓN

1.1 Módulos implicados

Este proyecto integra de forma coordinada los conocimientos adquiridos en la totalidad ciclo formativo de Desarrollo de Aplicaciones Multiplataforma:

Módulo	Contribución al proyecto
Programación	Lógica de negocio en Java 17; arquitectura por capas; POO con interfaces, records, Optional y colecciones
Bases de Datos	Diseño del modelo relacional normalizado; diagrama entidad-relación; consultas JPQL
Acceso a Datos	Integración ORM con JPA/Hibernate; repositorios Spring Data; gestión de transacciones
Desarrollo de Interfaces	Implementación del frontend con Angular 21; componentes reutilizables; modo oscuro/claro con Signals
Programación de Servicios y Procesos	Diseño y desarrollo de la API REST stateless; comunicación HTTP; arquitectura de red con Docker
Sistemas Informáticos	Configuración del entorno Docker; red virtual entre contenedores; WSL2; proxy inverso Nginx
Entornos de Desarrollo	Control de versiones con Git y GitHub; IntelliJ IDEA; Maven; Angular CLI; Docker CLI
Lenguajes de Marcas	HTML semántico en Angular; JSON como formato de intercambio en la API; validaciones de esquema
Programación Multimedia y Dispositivos Móviles	Aplicación Android con Flutter; InAppWebView avanzado; gestión de conectividad y splash nativo
Sistemas de Gestión Empresarial	El sistema actúa como CRM ligero para la gestión del negocio; análisis de soluciones comerciales existentes
Proyecto de Desarrollo de Aplicaciones	Integración de todas las fases: análisis, diseño, implantación y documentación completa
Itinerario personal para la empleabilidad	

1.2 Breve descripción del proyecto

CitaBella es un sistema de gestión integral diseñado específicamente para el sector de la peluquería y estética. El sistema digitaliza la operativa diaria de una peluquería real sustituyendo los métodos manuales, agenda de papel y mensajería instantánea, por una solución centralizada, segura y desplegable en cualquier entorno mediante contenedores Docker.

El proyecto se estructura en tres fases planificadas desde el inicio:

- *Fase 1 (presente entrega — MVP completo):*

Sistema de gestión de citas con ciclo de vida completo y detección de solapamientos, gestión de clientes, empleados, tratamientos y productos, API REST con autenticación JWT, frontend web Angular, aplicación móvil Flutter y despliegue

Docker con un único comando.

• *Fase 2 (trabajo futuro):*

Gestión de ventas e inventario. Control de stock con movimientos de entrada y salida y registro de transacciones asociadas a citas.

• *Fase 3 (trabajo futuro):*

Notificaciones automáticas a clientes mediante email y/o WhatsApp para recordatorios de cita, confirmaciones y felicitaciones de cumpleaños.

Esta planificación modular garantiza que el modelo de datos y la arquitectura del sistema, diseñados íntegramente desde la Fase 1, soporten las ampliaciones previstas sin requerir reestructuraciones. Las entidades, repositorios y enumeraciones de las Fases 2 y 3 ya están definidos en el código base.

Nota sobre la extensión: La presente memoria supera el rango de 30–40 páginas indicado en la normativa como orientativo.

Esto se debe a que el proyecto integra tres componentes técnicos diferenciados (backend, frontend y aplicación móvil), cada uno con su propia arquitectura, tecnologías y decisiones de diseño, además de los anexos normativamente obligatorios. Se ha priorizado la completitud técnica y la trazabilidad frente a la reducción artificial del contenido.

Las secciones de anexos Anexo A a H, páginas 48–59, representan aproximadamente un tercio de la extensión total.

1.2.1 Objetivo general

Desarrollar un sistema funcional que permita gestionar de forma digital las citas, clientes, empleados y servicios de una peluquería, mediante una aplicación web conectada a una base de datos, con acceso seguro por roles y una interfaz clara y usable.

1.2.2 Objetivos específicos

1. Diseñar un modelo de base de datos relacional normalizado que cubra todas las entidades del negocio en sus tres fases de desarrollo, evitando migraciones disruptivas posteriores.
2. Desarrollar una API REST completa con Spring Boot que implemente seguridad stateless mediante JWT y control de acceso granular por roles (ADMIN, EMPLOYEE, CLIENT).
3. Implementar un frontend Angular modular con lazy loading, sistema de autenticación con interceptores, guardas de ruta por rol y soporte completo para modo oscuro y claro.

4. Desarrollar una aplicación móvil Android con Flutter que envuelva la interfaz web en un contenedor InAppWebView nativo, gestionando conectividad, splash nativo y navegación.
5. Orquestar todos los componentes mediante Docker Compose, posibilitando el despliegue con un único comando.
6. Aplicar buenas prácticas: arquitectura por capas, DTOs como records Java, validaciones en dos niveles y manejo centralizado de errores.
7. Mantener trazabilidad del desarrollo mediante Git, GitHub Projects y commits descriptivos.

1.2.3 Uso del proyecto

Los perfiles de administrador y empleado gestionan la agenda de citas, el registro de clientes y el catálogo de servicios desde el panel privado. El perfil de cliente consulta sus propias citas. Cualquier visitante accede al catálogo de servicios y productos sin autenticación.

La aplicación móvil Flutter proporciona acceso completo al sistema desde dispositivos Android reutilizando íntegramente la lógica e interfaz ya desarrolladas.

1.2.4 Tecnologías investigadas no vistas en clase

Tecnología	Por qué se investigó	Valor diferencial
Spring Security + JWT stateless	El ciclo no cubre autenticación por tokens ni la cadena de filtros de Spring Security	Protege la API sin estado de sesión, esencial para clientes web y móviles
Angular Signals	Sistema de reactividad de Angular 17+, no cubierto en clase	Gestión del tema oscuro/claro de forma reactiva sin librerías externas
Springdoc OpenAPI	Generación automática de documentación API, no vista en clase	Produce Swagger UI interactivo directamente desde anotaciones del código
Flutter + InAppWebView	El ciclo introduce Android nativo, no Flutter	Solución multiplataforma que reutiliza la SPA Angular como app nativa
Nginx como proxy inverso	Configuración de servidores web no vista en profundidad	Punto de entrada único: sirve Angular y redirige /api al backend

2. ESTUDIO PREVIO

2.1 Análisis del caso real

El proyecto se basa en el análisis de los procesos operativos de una peluquería real, "*Ruth Molina*", situada en Villar del Arzobispo (Valencia). Tras una entrevista inicial con la propietaria se identificaron las siguientes problemáticas concretas en la gestión actual del negocio:

- Gestión manual de citas: la agenda física no permite consultas rápidas ni detecta conflictos horarios. La modificación o cancelación de citas no siempre se propaga correctamente, generando solapamientos no detectados.
- Comunicación no estructurada: la coordinación con clientes se realiza íntegramente por WhatsApp, sin registro de confirmaciones ni historial de comunicaciones.
- Ausencia de historial de cliente: la información está dispersa entre la agenda física y los contactos del teléfono. No existe un registro centralizado de visitas previas ni preferencias.
- Sin control de solapamientos: ningún mecanismo impide asignar dos citas al mismo empleado en la misma franja horaria.
- Visibilidad digital limitada: el negocio no dispone de presencia web propia donde los clientes puedan consultar servicios o disponibilidad.

[VER ANEXO A — EXTRACTO DE LA ENTREVISTA CON LA PROPIETARIA]

2.2 Necesidades detectadas

A partir del análisis anterior se priorizaron las siguientes necesidades funcionales:

#	Necesidad	Prioridad	Fase
1	Registro y gestión centralizada de citas con control de solapamientos	Alta	Fase 1
2	Ficha digital de cliente con datos de contacto y género	Alta	Fase 1
3	Control de acceso diferenciado por rol (admin, empleado, cliente)	Alta	Fase 1
4	Catálogo de servicios y productos accesible públicamente	Media	Fase 1
5	Aplicación móvil para gestión desde el teléfono	Media	Fase 1
6	Gestión de inventario y control de stock de productos	Media	Fase 2
7	Registro de ventas asociadas a citas	Media	Fase 2
8	Notificaciones automáticas de recordatorio de citas	Baja	Fase 3

2.3 Estudio de soluciones existentes

Se han analizado las principales soluciones comerciales disponibles en el mercado:

Criterio	Treatwell	Fresha	Booksy	SimplyBook.me	CitaBella
Modelo de negocio	Comisión por reserva	Freemium + suscripción	Suscripción mensual	Suscripción mensual	Propiedad total
Coste aproximado	20–30% por reserva	Desde 0 € (limitado)	~30 €/mes	~10–30 €/mes	Coste de desarrollo único
Personalización visual	Baja	Baja	Baja	Media	Total
Portabilidad de datos	No	No	No	Exportación limitada	Total
Adaptación al negocio	Genérica	Genérica	Genérica	Configurable	A medida
Dependencia externa	Total	Total	Total	Total	Ninguna
Funcionalidades innecesarias	Muchas	Muchas	Muchas	Bastantes	Solo las necesarias

Conclusión: las soluciones existentes están orientadas a negocios que necesitan reservas online desde el exterior, lo que introduce costes recurrentes, dependencia de terceros y funcionalidades no requeridas. CitaBella adopta la estrategia de sistema propio, sin comisiones, con datos bajo control total y capacidad de crecimiento modular.

2.4 Propuesta de valor y público objetivo

Propuesta de valor: CitaBella ofrece a una peluquería pequeña-mediana la posibilidad de gestionar su operativa con un sistema propio, sin comisiones, sin dependencia de plataformas externas y con datos completamente bajo su control, desplegable con un único comando en cualquier equipo.

Público objetivo primario: administradores y empleados de peluquerías de tamaño pequeño-mediano (1–10 empleados) que actualmente gestionan su actividad con métodos manuales o herramientas de mensajería.

Público objetivo secundario: clientes del negocio que acceden para consultar sus citas o el catálogo de servicios.

2.5 Análisis DAFO

Factores positivos		Factores negativos
Factores internos	FORTALEZAS	DEBILIDADES
	Stack moderno y bien integrado (Spring Boot 3 + Angular 21 + Flutter + Docker)	Proyecto individual: menor velocidad de desarrollo, sin revisión por pares
	Arquitectura desacoplada y modular	Falta de feedback directo del cliente durante la etapa de desarrollo
	Despliegue reproducible con Docker Compose	Sin tests automatizados en la entrega del MVP
	Modelo de datos diseñado para las tres fases desde el inicio	Sin HTTPS configurado en el entorno de desarrollo local
	Documentación técnica completa en el repositorio	Dependencia de conexión de red en la app móvil (sin modo offline)
Factores externos	OPORTUNIDADES	AMENAZAS
	Mercado de peluquerías con alta proporción de negocios sin digitalizar	Soluciones gratuitas (Fresha) pueden cubrir necesidades básicas sin coste
	Creciente demanda de soluciones sin dependencia de plataformas externas	Coste de mantenimiento técnico si no hay soporte disponible
	La arquitectura permite evolucionar a SaaS multi-tenant	Plataformas comerciales consolidan usuarios con marketing integrado
	Fases 2 y 3 añaden valor diferencial no cubierto en el MVP	Sin acceso a tiendas de apps en la entrega académica

2.6 Estrategia derivada del DAFO

Las fortalezas técnicas del proyecto compensan directamente las debilidades operativas de las soluciones existentes. La estrategia adoptada es la de diferenciación por control y adaptación: frente a plataformas genéricas con modelo de suscripción o comisión, CitaBella ofrece propiedad total del sistema y los datos, adaptación exacta al flujo de trabajo del negocio y capacidad de evolución autónoma sin cambiar de proveedor.

3. PLAN DE TRABAJO

3.1 Metodología

El proyecto se ha desarrollado de forma incremental, avanzando por fases que van desde el análisis hasta la integración.

- Entregas parciales funcionales: cada fase de desarrollo produjo un artefacto ejecutable y verificable antes de avanzar. El diseño del modelo de datos se completó antes de iniciar el backend.

- Revisión continua del alcance: el alcance del MVP se ajustó durante el desarrollo para garantizar la entrega de un sistema completo y funcional dentro del plazo.

Control de versiones como registro de progreso: los commits con mensajes descriptivos reflejan el avance real del trabajo.

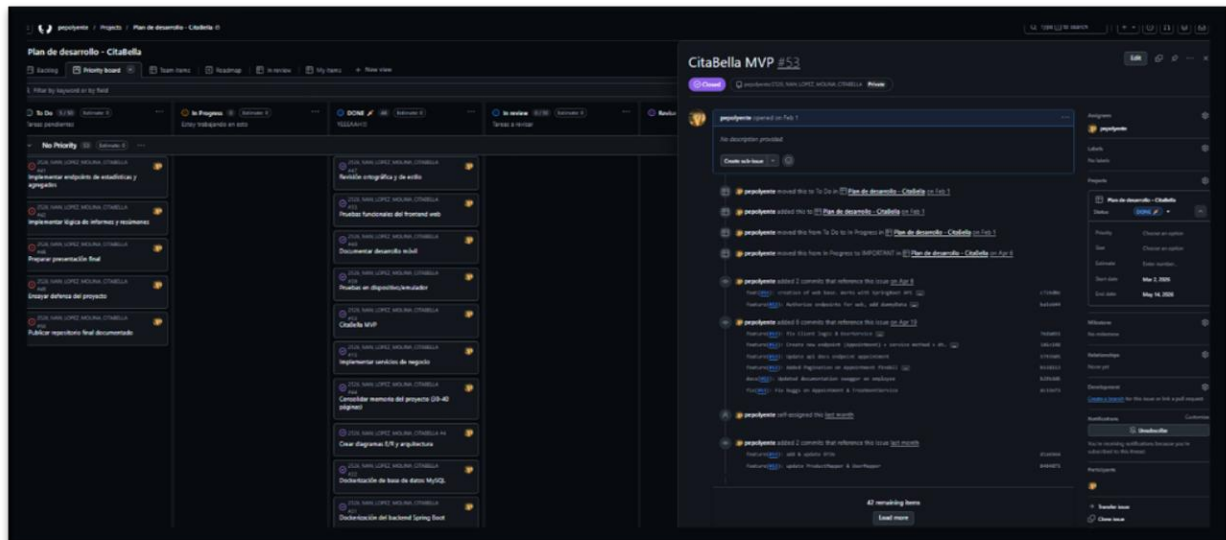


Ilustración 1 Priority Board Git Hub Projects

3.2 Temporalización

#	Etapas	Necesidad	Fase
1	Análisis	Entrevista, identificación de necesidades, estudio de mercado, definición del alcance	Octubre 2025 – Noviembre 2025
2	Diseño	Modelo de datos, arquitectura, API REST, wireframes conceptuales	Noviembre 2025 – Enero 2026
3	Desarrollo backend	Entidades JPA, repositorios, servicios, controladores, seguridad JWT	Diciembre 2025 – Abril 2026
4	Desarrollo frontend	Módulos Angular, servicios HTTP, componentes, zona pública, temas	Febrero 2026 – Mayo 2026
5	Desarrollo móvil	Flutter, InAppWebView, splash, conectividad	Mayo 2026
6	Integración y despliegue	Docker Compose, Nginx, proxy, verificación end-to-end	Diciembre 2025 – Mayo 2026
7	Pruebas y ajustes	Pruebas manuales con Bruno, validación de historias de usuario	Enero 2026 – Mayo 2026
8	Documentación	Memoria, diagramas, capturas, README	Octubre 2025 – Mayo 2026

Las fechas de cada fase se estimaron a partir del histórico de commits, issues y tareas gestionadas mediante GitHub Projects y control de versiones Git.

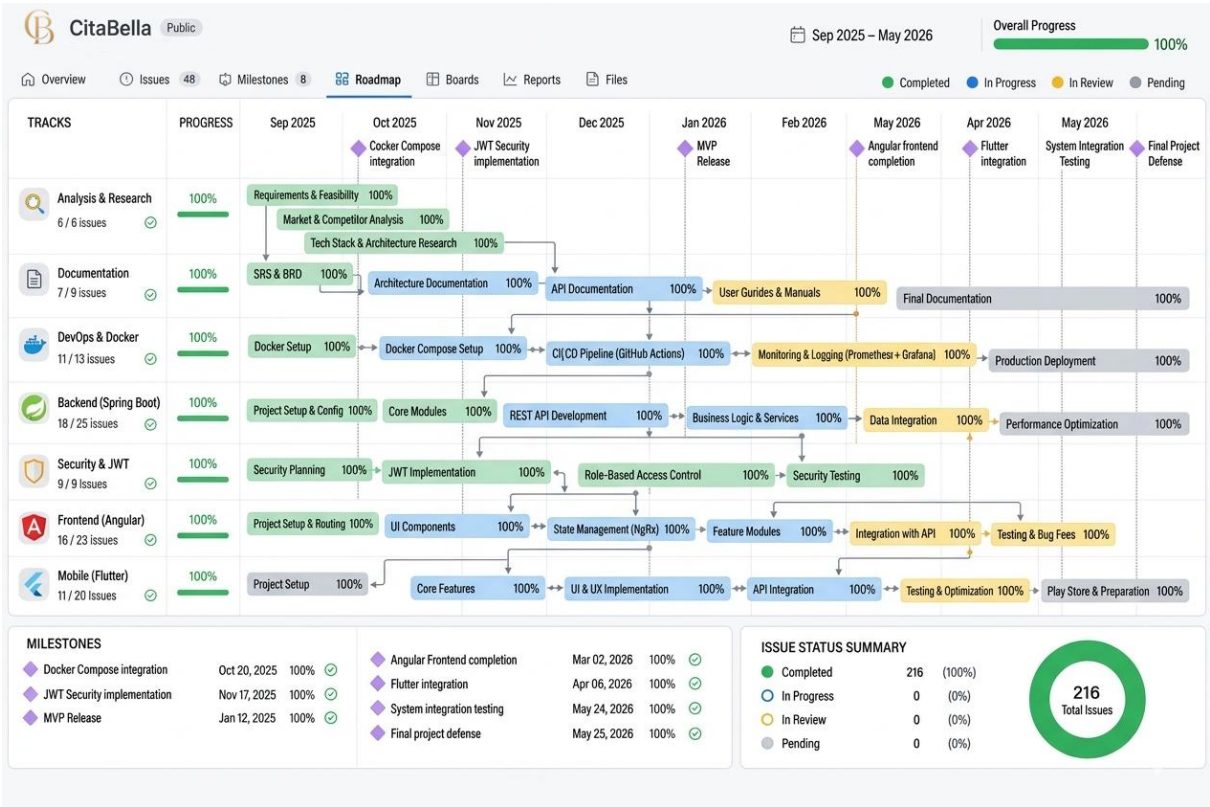


Ilustración 2 Git Hub Projects RoadMap Representación

3.3 Herramientas de seguimiento

Herramienta	Uso en el proyecto
Git	Control de versiones local y remoto
GitHub	Alojamiento del repositorio público, historial de commits, gestión de ramas
GitHub Projects	Tablero Kanban para seguimiento de tareas

Repositorio público del proyecto:

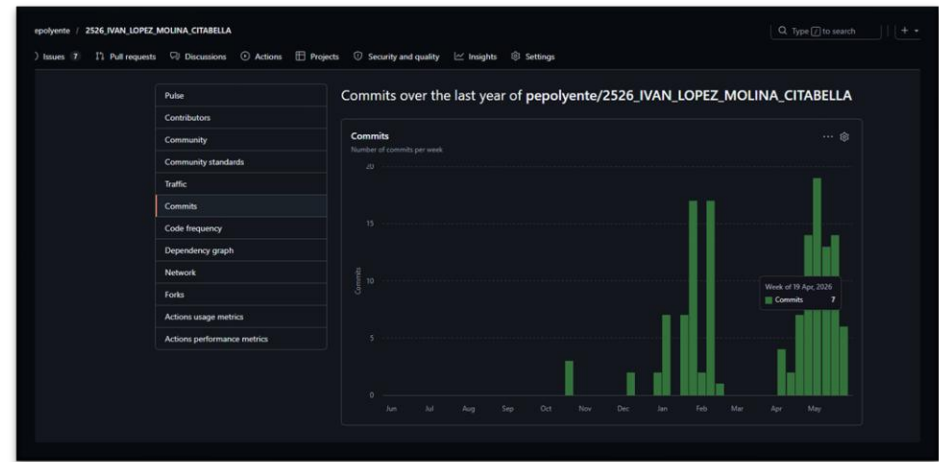


Ilustración 3 Git Hub Commits over the last year

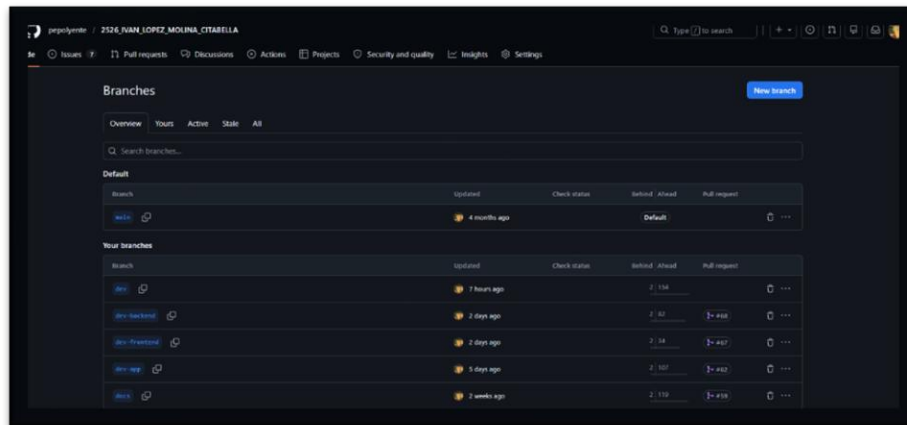


Ilustración 4 Git Hub branches

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA

4. DISEÑO

4.1 Diseño general (análisis)

4.1.1 Diagrama de contexto

El sistema CitaBella interactúa con los siguientes actores externos:

- **Administrador:** acceso completo al sistema. Gestiona citas, clientes, empleados, tratamientos, usuarios del sistema y tiene visibilidad sobre toda la actividad del negocio.
- **Empleado:** acceso a la gestión operativa diaria. Crea y consulta citas, gestiona la ficha de clientes.
- **Cliente:** usuario registrado que puede consultar sus propias citas desde el panel privado.
- **Visitante (sin autenticación):** cualquier persona que accede a la zona pública del sistema para consultar servicios y productos.

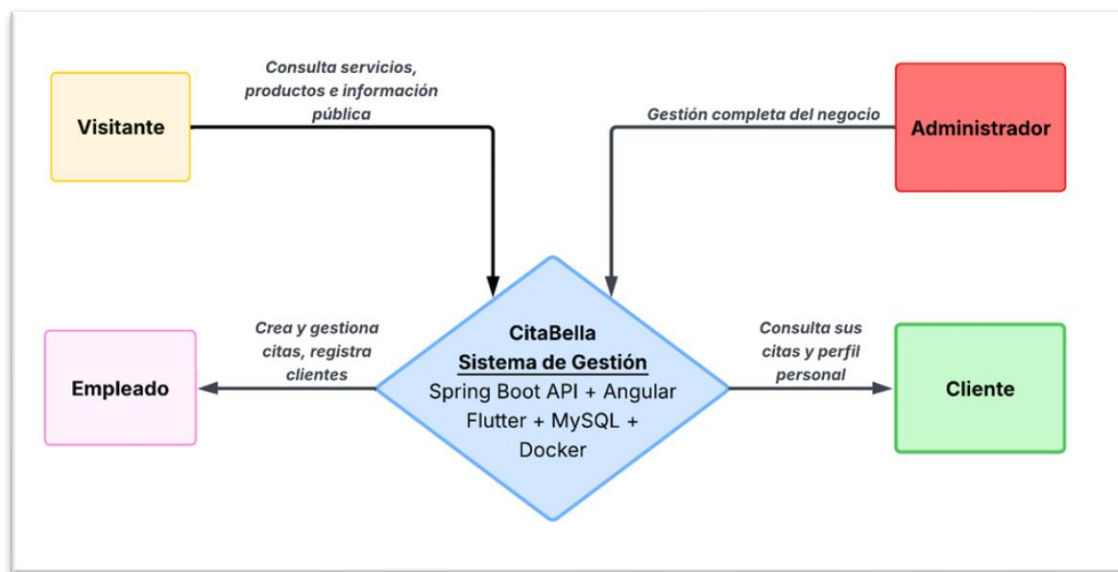


Ilustración 5 Diagrama de contexto

4.1.2 Historias de usuario

El sistema se ha especificado mediante 28 historias de usuario que cubren las tres fases de desarrollo planificadas. Las historias siguen el formato estándar con frente (ID, título, reglas de negocio) y dorso (criterios de aceptación en formato Given/When/Then).

Las 20 historias implementadas en el MVP se distribuyen por área funcional:

ID	Título (resumen)	Actor	Área
HU-01	Iniciar sesión con usuario y contraseña	Usuario registrado	Autenticación
HU-02	Registrarse como visitante (cuenta pendiente de activación)	Visitante	Autenticación
HU-03	Ver la página principal con tratamientos y productos destacados	Visitante	Zona pública
HU-04	Consultar el catálogo completo de tratamientos	Visitante	Zona pública
HU-05	Ver el catálogo de productos con imagen y precio de venta	Visitante	Zona pública
HU-06	Crear una cita asignando cliente, empleado, tratamientos y horario	Empleado / Admin	Citas
HU-07	Ver el calendario de citas en vista mensual, semanal y diaria	Empleado / Admin	Citas
HU-08	Cambiar el estado de una cita según la máquina de estados	Empleado / Admin	Citas
HU-09	Cancelar una cita liberando la franja horaria	Empleado / Admin	Citas
HU-10	Registrar y gestionar clientes con ficha digital	Empleado / Admin	Clientes
HU-11	Vincular un cliente existente a una cuenta de usuario	Admin	Clientes / Usuarios
HU-12	Registrar y gestionar empleados activos del negocio	Admin	Empleados
HU-13	Gestionar el catálogo de tratamientos	Admin	Tratamientos
HU-14	Gestionar productos con precio de compra, venta, tipo de uso y proveedor	Admin	Productos
HU-15	Ver las propias citas desde el panel privado como cliente	Cliente	Mis citas
HU-16	Ver y gestionar todos los usuarios del sistema con control de roles	Admin	Administración
HU-17	Cambiar entre modo claro, oscuro y automático en la interfaz	Cualquier usuario	Interfaz
HU-18	Cerrar sesión eliminando el token	Usuario autenticado	Autenticación
HU-19	Acceder al sistema desde la aplicación Android instalada	Empleado / Cliente	App móvil
HU-20	Explorar y probar la API mediante Swagger UI	Desarrollador / Admin técnico	API

Las 8 historias de las Fases 2 y 3 (HU-21 a HU-28) cubren la gestión de ventas, stock, alertas, informes y notificaciones automáticas. Sus entidades ya están modeladas en la base de datos.

[VER ANEXO E — HISTORIAS DE USUARIO COMPLETAS CON CRITERIOS DE ACEPTACIÓN EN FORMATO GHERKIN]

4.1.3 Flujos de usuario

Flujo de autenticación (HU-01, HU-02, HU-18)

El visitante puede registrar una cuenta nueva, que queda en estado *PENDING* hasta vinculación. Un usuario registrado introduce credenciales; el backend valida con *BCrypt*, genera un *JWT HS256* y lo devuelve. El frontend almacena el token en *localStorage* y redirige según rol: *ADMIN/EMPLOYEE* a */panel/appointments*, *CLIENT*

a /panel/my-appointments. Al cerrar sesión el token se elimina y se redirige a la zona pública.

Flujo de gestión de citas (HU-06, HU-07, HU-08, HU-09)

Desde el calendario, un empleado o administrador crea una nueva cita mediante el asistente de tres pasos: (1) cliente y horario, (2) tratamientos y empleado, (3) confirmación. La cita se crea con estado *PENDING*. Si la franja se solapa con otra cita del mismo empleado, el sistema marca *hasOverlap = true* y muestra un indicador  sin bloquear la creación. La cita transita por su ciclo de vida a través del modal de detalle.

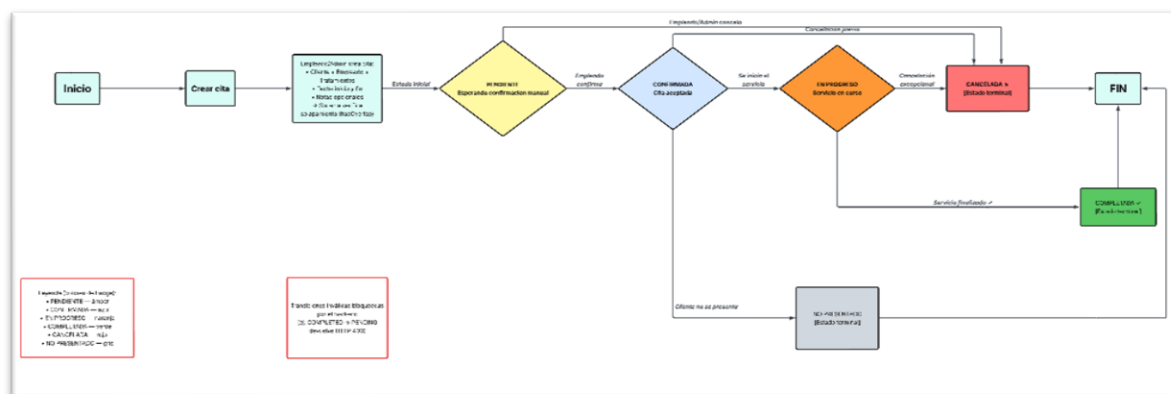


Ilustración 6 Diagrama de flujo de citas

Flujo de zona pública (HU-03, HU-04, HU-05)

Cualquier visitante sin autenticación accede a la página principal con tratamientos y productos destacados, al catálogo completo de servicios con duración y precio, y al catálogo de productos con imagen y precio de venta. Solo se muestran entidades con *active = true*. Los productos de tipo *INTERNAL* no son visibles al exterior.

Flujo de la aplicación móvil (HU-19)

Al abrir la app: splash nativo → verificación de conectividad → si hay red y el servidor responde, la SPA Angular se carga en el WebView con barra de progreso real → overlay hace fade out → app operativa. Si no hay red: pantalla "Sin conexión". Si hay red pero el servidor no responde en 8 segundos: pantalla "Servidor en mantenimiento".

4.1.4 Modelo conceptual

Las entidades principales del sistema y sus relaciones son las siguientes:

- Usuario: cuenta de acceso con rol (ADMIN, EMPLOYEE, CLIENT, USER) y estado (PENDING, ACTIVE, LOCKED). Puede estar vinculado opcionalmente a un perfil de cliente o empleado, pero no a ambos simultáneamente.

- Cliente: persona que recibe servicios. Tiene nombre, teléfono único, fecha de nacimiento y género. Puede estar vinculado a una cuenta de usuario.
- Empleado: trabajador del negocio con nombre único y puesto. Puede estar vinculado a una cuenta de usuario.
- Tratamiento: servicio ofrecido con nombre único, descripción, duración mínima/máxima y precio.
- Cita: entidad central. Relaciona un cliente, un empleado y uno o varios tratamientos, con una franja horaria y un estado que sigue la máquina de estados definida.
- Producto: artículo del catálogo con nombre único, categoría, precio de compra y de venta, tipo de uso (INTERNAL/SALE/BOTH) y flag de criticidad para alertas futuras.
- Venta: transacción comercial que puede asociarse a una cita, con detalle de tratamientos y productos al precio del momento. Implementación prevista en Fase 2.
- Notificación: mensaje automático programado (recordatorio, confirmación, cumpleaños). Implementación prevista en Fase 3.

4.2 Diseño detallado (diseño técnico)

4.2.1 Diagrama Entidad-Relación de la Base de Datos

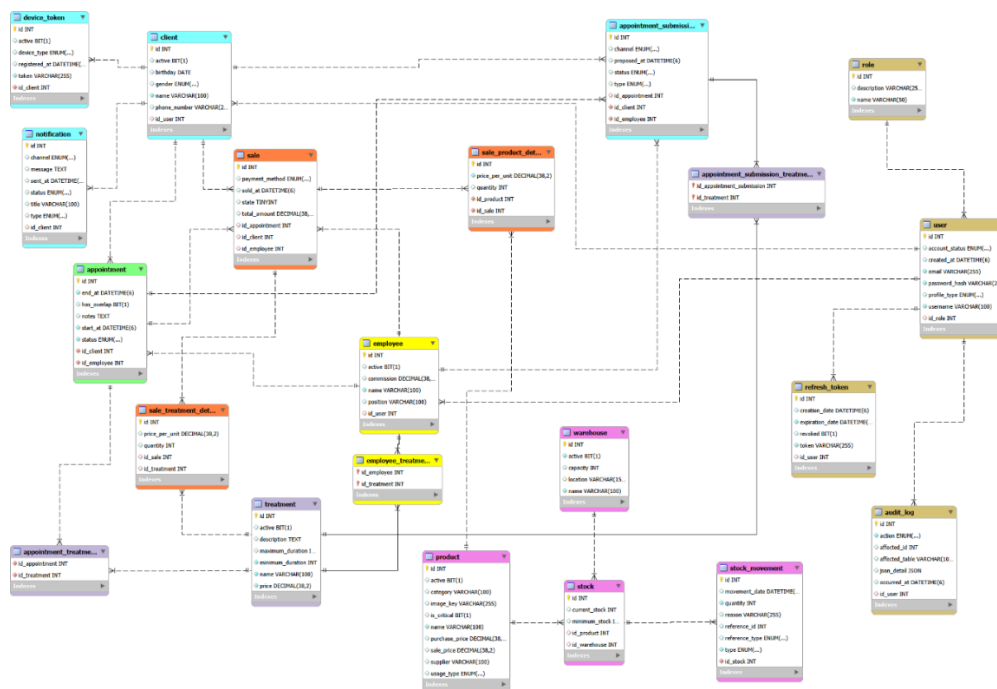


Ilustración 7 Diagrama ER base de datos

DOCUMENTACIÓN TÉCNICA DETALLADA DEL MODELO DE DOMINIO DISPONIBLE EN:

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA/blob/main/docs/Backend.md (SECCIÓN 9)

4.2.2 Diagrama de clases (Backend)

El backend sigue una arquitectura por capas claramente definida. Las capas son, de menor a mayor nivel de abstracción:

- Entidades JPA (entity): clases Java que mapean directamente con las tablas de la base de datos mediante anotaciones de Jakarta Persistence.
- Repositorios (repository): interfaces que extienden JpaRepository y permiten acceder a los datos sin escribir SQL manualmente. Incluyen consultas personalizadas donde es necesario.
- Servicios (service): contienen la lógica de negocio. Se definen mediante interfaces y se implementan en clases separadas, lo que facilita el mantenimiento y las pruebas.
- Controladores (controller): exponen los endpoints REST y delegan en los servicios. Se encargan únicamente de recibir la petición, validarla y devolver la respuesta adecuada.
- DTOs (dto): objetos de transferencia de datos implementados como records de Java. Separan la representación interna de las entidades de lo que se expone en la API.
- Mappers : clases estáticas que convierten entre entidades y DTOs, manteniendo el código de transformación centralizado y reutilizable.

DOCUMENTACIÓN TÉCNICA COMPLETA DE LA ARQUITECTURA POR CAPAS EN:

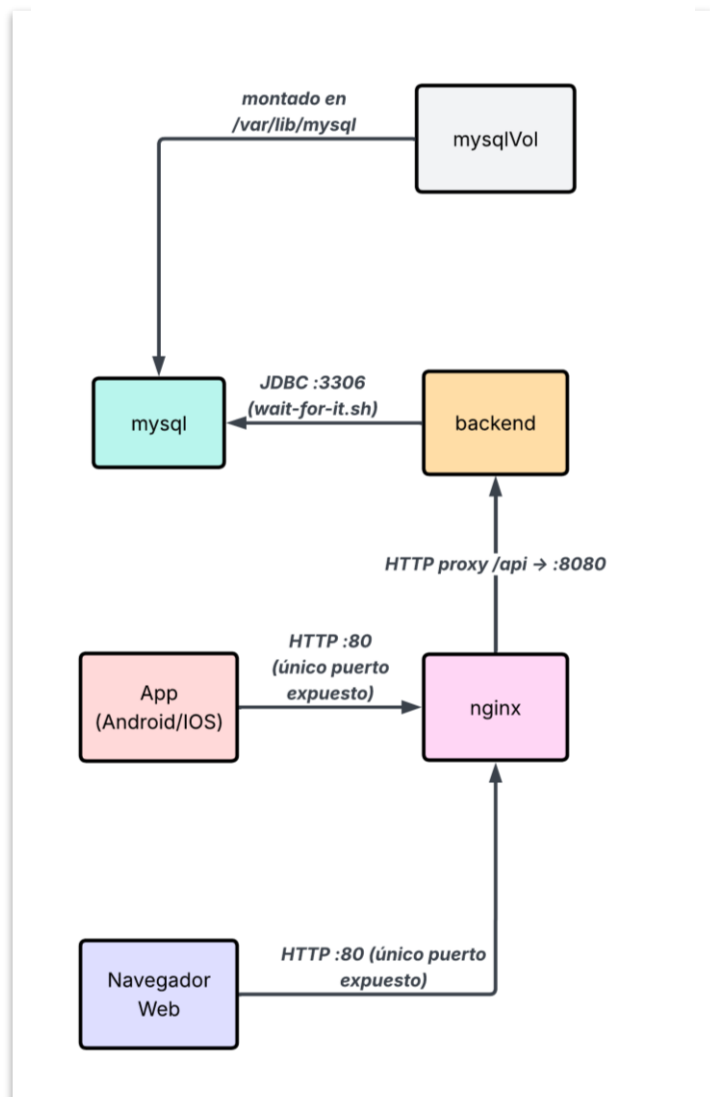
https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA/blob/main/docs/Backend.md

4.2.3 Arquitectura del sistema

El sistema sigue una arquitectura cliente-servidor distribuida con separación clara entre frontend y backend:

- El backend expone una API REST sobre el puerto 8080 y gestiona toda la lógica de negocio, la validación de datos y el acceso a la base de datos MySQL.
- El frontend Angular se sirve en desarrollo desde el puerto 4200. En producción, Nginx actúa como servidor web y proxy inverso, sirviendo los ficheros estáticos del frontend y redirigiendo las peticiones a /api hacia el backend.
- La base de datos MySQL corre en su propio contenedor Docker, aislada del resto del sistema.
- La comunicación entre frontend y backend es exclusivamente mediante HTTP con JSON, autenticada con tokens JWT en la cabecera Authorization.

Ilustración 8 Diagrama Arquitectura del Sistema



El diagrama de arquitectura se incluye en el Anexo D.

4.2.4 Diseño de la API REST

La API REST expone 46 endpoints organizados en 8 grupos funcionales. La documentación interactiva completa, generada automáticamente mediante Springdoc OpenAPI, está disponible en `/swagger-ui.html` durante la ejecución del sistema y en formato estático en `docs/api-docs.html` del repositorio. La siguiente tabla recoge los grupos de endpoints con su conteo y nivel de acceso mínimo:

Grupo	Endpoints	Acceso mínimo
Autenticación (/api/auth)	3	Público
Citas (/api/appointments)	7	EMPLOYEE
Clientes (/api/clients)	7	EMPLOYEE
Empleados (/api/employees)	7	EMPLOYEE
Tratamientos (/api/treatments)	7	ADMIN
Productos (/api/products)	7	Público (lectura)
Usuarios (/api/users)	7	ADMIN
Roles (/api/roles)	1	ADMIN

Update appointment (reschedule)

Requires authentication. Allowed roles: ADMIN, EMPLOYEE.

AUTHORIZATIONS: *bearerAuth*

REQUEST BODY SCHEMA: *application/json*

required

- id** integer <int32>
- employeeId** integer <int32>
- treatmentsIds** Array of integers <int32> *unique* [items <int32 >]
- startAt** string <date-time>
- endAt** string <date-time>
- notes** string

Responses

Request samples

Payload

Content type: *application/json*

```
{
  "id": 0,
  "employeeId": 0,
  "treatmentsIds": [
    0
  ],
  "startAt": "2019-08-24T14:15:22Z",
  "endAt": "2019-08-24T14:15:22Z",
  "notes": "string"
}
```

Ilustración 9 Api Docs HTML Endpoints

4.2.5 Vistas implementadas del frontend

Se han diseñado y desarrollado las siguientes vistas principales del sistema:

Zona pública (sin autenticación):

- Página de inicio (/): hero con glow effects y glassmorfismo, tratamientos destacados (máx. 4) y productos destacados (máx. 4). Skeleton loading durante la carga.

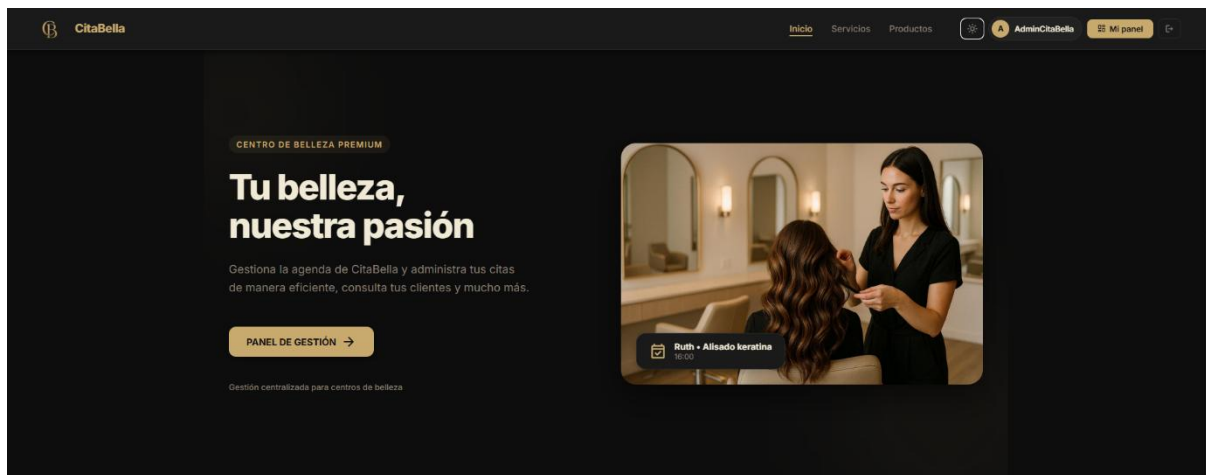


Ilustración 10 Pantalla principal CitaBella Web

- Página de servicios (/servicios): grid con todos los tratamientos activos, nombre, duración mínima y precio.
- Página de productos (/productos): galería de productos activos con imagen (fallback a imagen por defecto si no tiene), nombre y precio de venta.

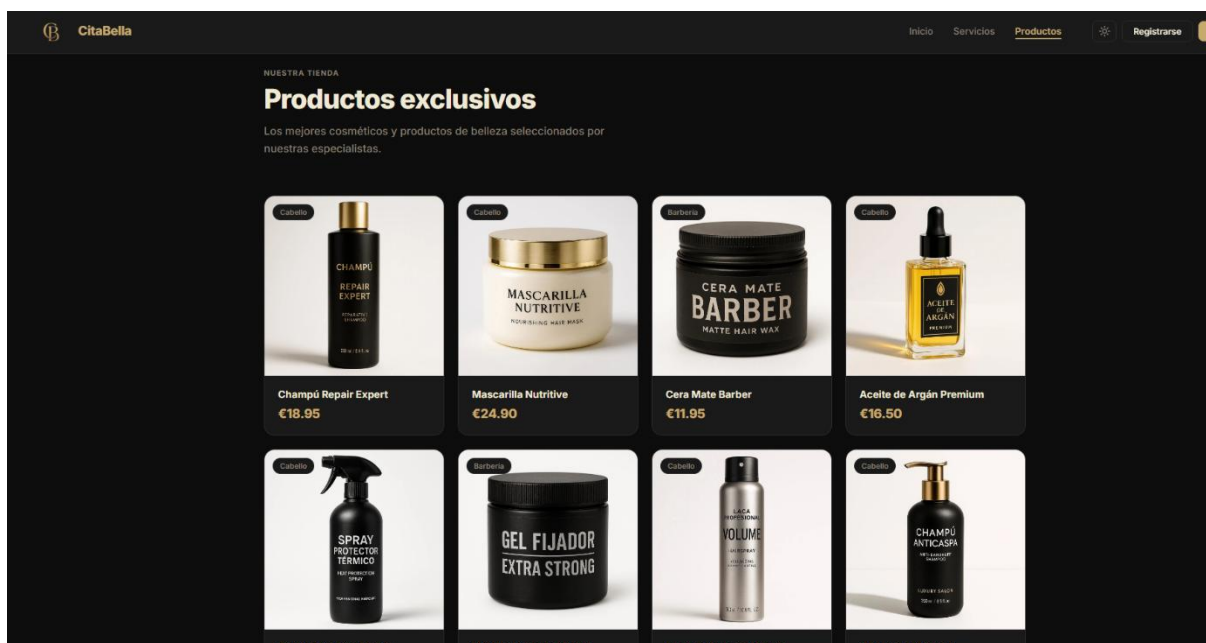


Ilustración 11 Pantalla Productos CitaBella Web

Autenticación:

- Login (/login): formulario centralizado sobre fondo degradado oscuro, con opción mostrar/ocultar contraseña.
- Registro (/register): formulario con validación de email único, username único y contraseña mínima de 6 caracteres.

Panel privado:

- Citas (/panel/appointments): calendario FullCalendar con vistas mes/semana/día, colores por estado, indicador para solapamientos, y wizard de 3 pasos para crear citas.

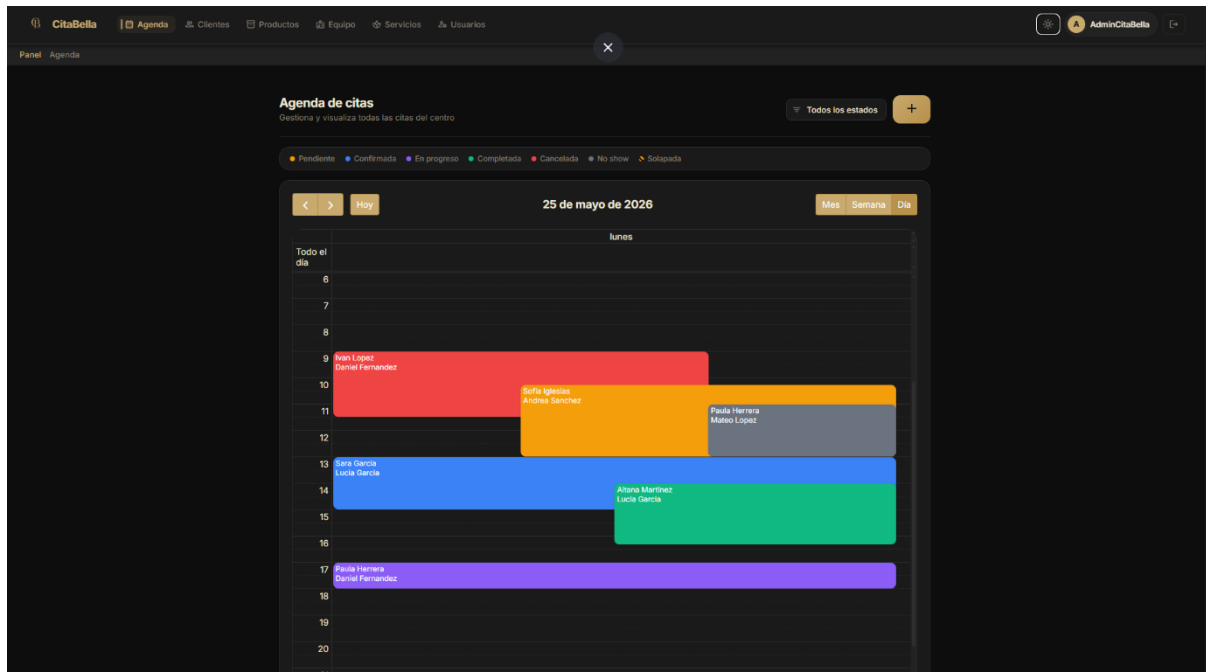


Ilustración 12 Pantalla Agenda de Citas CitaBella WebVista Semanal

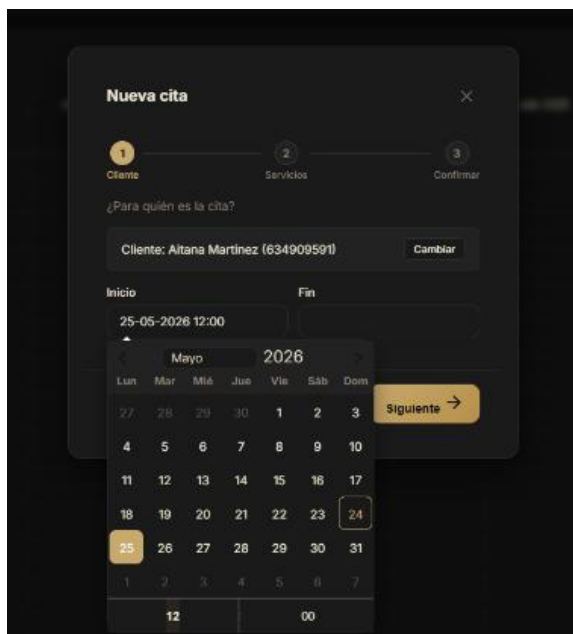


Ilustración 13 Vista Wizard creación de cita + flatpick

Nueva cita	
Cliente	Aitana Martínez
Empleado	Lucía García
Servicios	Alisado keratina, Peinado boda

Ilustración 14 Vista Wizard creación de cita 3 PASOS

- Clientes (/panel/clients): tabla paginada con búsqueda, edición inline, vinculación/desvinculación de usuario.
- Empleados (/panel/employees): tabla con búsqueda, edición inline y activación/desactivación.
- Tratamientos (/panel/treatments): tabla paginada con búsqueda y edición inline.
- Productos (/panel/products): tabla paginada con búsqueda, edición inline y formulario colapsable.
- Usuarios (/panel/admin): tabla paginada con cambio de rol inline, bloqueo/activación.

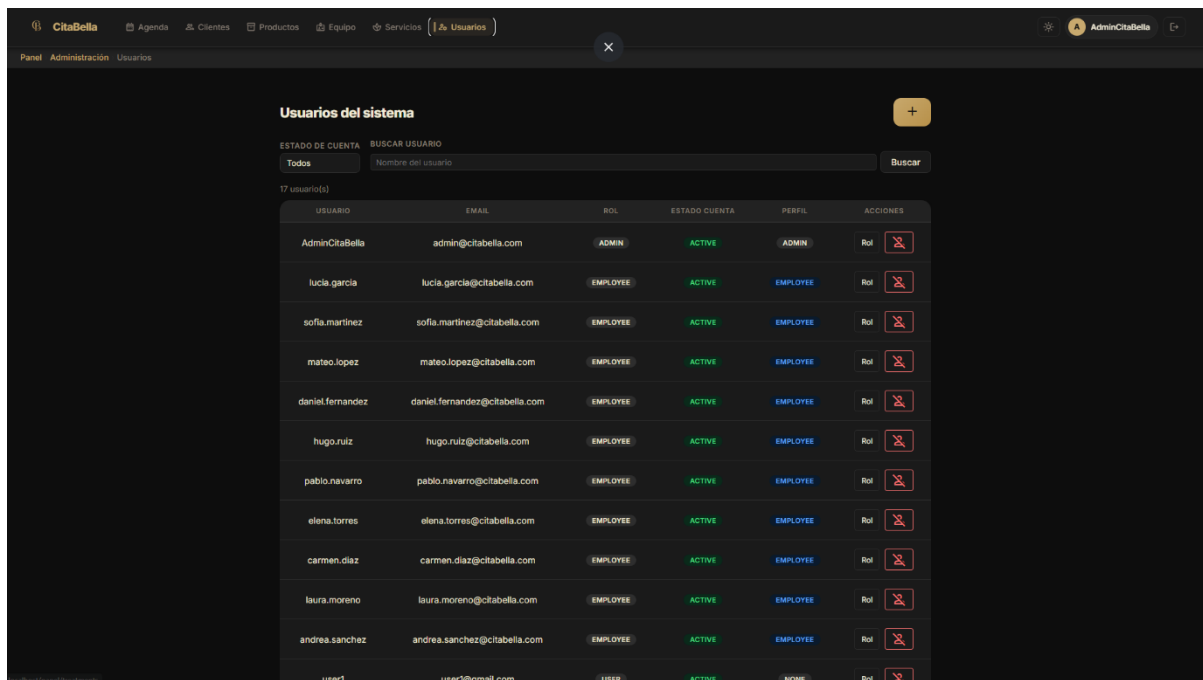


Ilustración 15 Pantalla Gestión Usuarios CitaBella Web

- Mis citas (/panel/my-appointments): calendario de solo lectura para el perfil CLIENT.

El sistema incluye soporte para modo oscuro (Gold — acento #C8A96E) y claro (Navy — primario #0A0F1E), activable manualmente o de forma automática según la preferencia del sistema operativo.

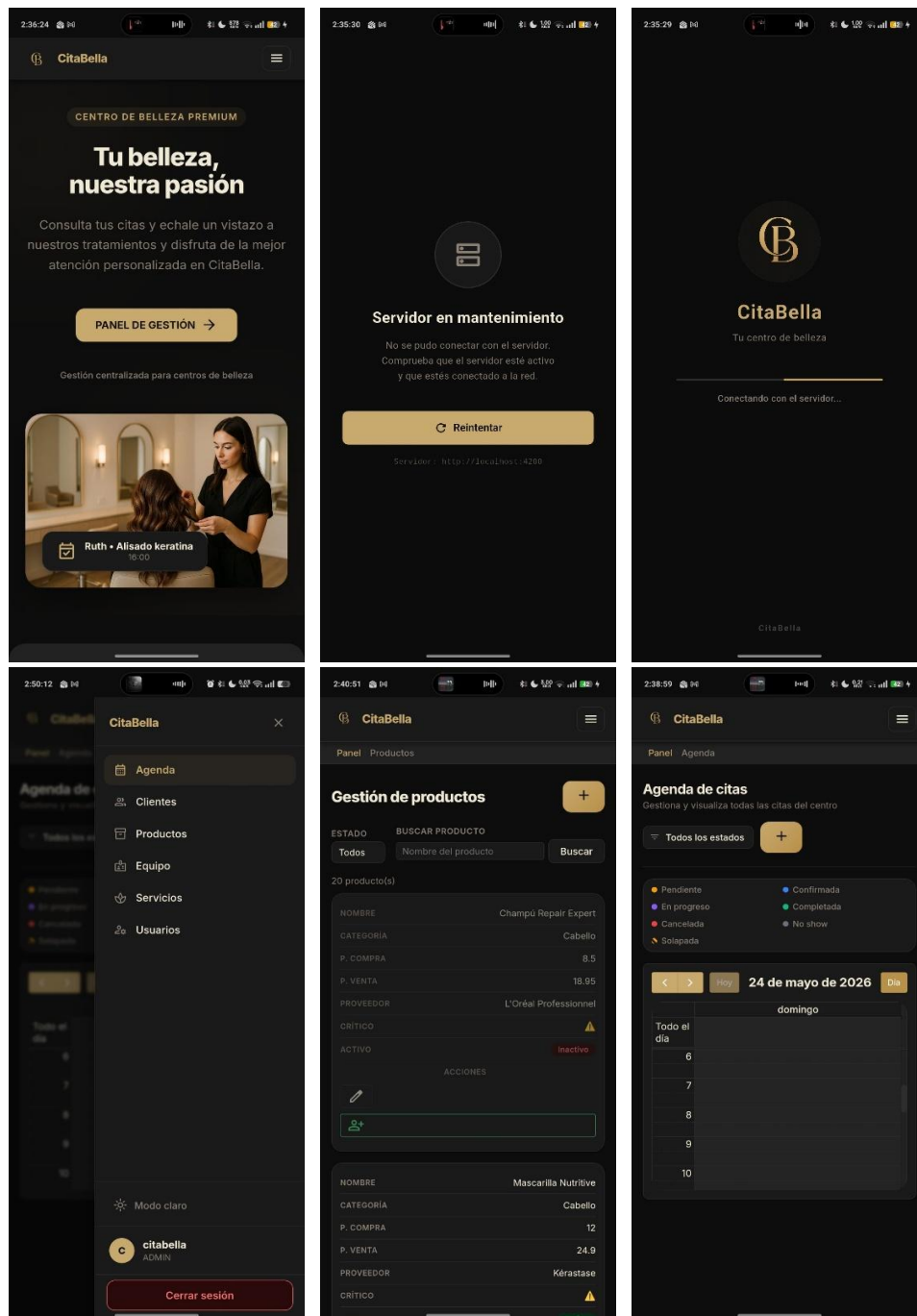
DOCUMENTACIÓN TÉCNICA COMPLETA DEL FRONTEND EN:

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA/blob/main/docs/Frontend.md

4.2.6 Navegación de la App Android (Flutter)

La aplicación móvil actúa como un shell nativo Flutter que envuelve la SPA Angular mediante flutter_inappwebview. No reimplementa la interfaz ni la lógica de negocio. Flutter gestiona exclusivamente la capa nativa: splash screen, estados de conectividad, tema visual Material 3 y navegación con el botón Atrás de Android.

Ilustraciones 16, 17, 18, 19, 20, 21 Flutter WebView + Splash + Conectividad



El flujo de pantallas nativas es: splash nativo → splash Flutter (animación fade+scale, 1800 ms) → verificación de conectividad → WebView con barra de progreso real → Angular operativa. Si hay fallo de red o servidor: pantalla de error con botón Reintentar.

DOCUMENTACIÓN TÉCNICA COMPLETA DE LA APP MÓVIL EN:

https://github.com/pepoliente/2526_IVAN_LOPEZ_MOLINA_CITABELLA/blob/main/docs/App-Android-IOS.md

El sistema se despliega mediante Docker Compose con tres servicios contenerizados comunicados a través de la red interna app-net:

- **mysql** (MySQL 8): base de datos relacional. Datos persistidos en el volumen Docker mysql_data. No expuesto al exterior.
- **backend** (Spring Boot): API REST construida mediante Dockerfile multietapa (compilación Maven + imagen de ejecución JRE ligera). No expuesto al exterior.
- **nginx** (Nginx Alpine): sirve los archivos estáticos del frontend Angular compilado y actúa como proxy inverso redirigiendo /api al backend. Único servicio expuesto, en el puerto 80.

```
shdwdrk@DESKTOP-0920L4K:~/Projects/citabella-dev-test$ docker compose up -d
[+] Running 3/4
 ✓ Network citabella-dev-test_app-net    Created
 ✓ Container citabella-mysql             Started
  ⋮ Container citabella-backend          Waiting
 ✓ Container citabella-nginx             Created
```

Ilustración 22 docker compose up -d

```
citabella-backend | X Esperando a MySQL en mysql:3306...  
citabella-mysql    | 2026-05-24T22:27:S1.345980Z 0 [System] [MY-010910] [Server] /usr/sbin/mysqld: Shutdown complete (mysqld 8.4.9) MySQL Community Server - GPL.  
citabella-mysql    | 2026-05-24T22:27:S1.345999Z 0 [System] [MY-015016] [Server] MySQL Server - end.  
citabella-mysql    | 2026-05-24 22:27:S1+00:00 [Note] [Entrypoint]: Temporary server stopped  
  
citabella-mysql    | 2026-05-24 22:27:S2+00:00 [Note] [Entrypoint]: MySQL init process done. Ready for start up.  
  
citabella-backend | X Esperando a MySQL en mysql:3306...  
citabella-mysql    | 2026-05-24T22:27:S2.019585Z 0 [System] [MY-015015] [Server] MySQL Server - start.  
citabella-mysql    | 2026-05-24T22:27:S2.362984Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.4.9) starting as process 1  
citabella-mysql    | 2026-05-24T22:27:S2.371574Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.  
citabella-mysql    | 2026-05-24T22:27:S2.744341Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.  
citabella-mysql    | 2026-05-24T22:27:S3.022535Z 0 [Warning] [MY-010048] [Server] CA certificate ca.pem is self signed.  
citabella-mysql    | 2026-05-24T22:27:S3.022586Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported.  
citabella-mysql    | 2026-05-24T22:27:S3.026160Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysql' in the path is ac  
ry,  
citabella-mysql    | 2026-05-24T22:27:S3.052985Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port: 33060, socket: /var/run/my  
citabella-mysql    | 2026-05-24T22:27:S3.053263Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.4.9' socket: /var/run/mysqld/  
citabella-backend | ✓ MySQL disponible - arrancando backend
```

The Spring Boot logo consists of two stylized green wings forming a V-shape, positioned above a horizontal bar containing several small black circles.

```
:: Spring Boot ::                (v3.5.7)
```

Ilustración 23 docker compose logs

```

citabella-backend | 2026-05-24T22:28:05.501Z INFO 1 --- [CitaBellaAPI] [ main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 1 endpoint beneath base path '/actuator'
citabella-backend | 2026-05-24T22:28:06.551Z INFO 1 --- [CitaBellaAPI] [ main] r$InitializeUserDetailsManagerConfigurer : Global AuthenticationManager configured with UserDetailsServ
ice
citabella-backend | 2026-05-24T22:28:07.248Z INFO 1 --- [CitaBellaAPI] [ main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
citabella-backend | 2026-05-24T22:28:07.269Z INFO 1 --- [CitaBellaAPI] [ main] c.c.c.CitaBellaApiApplication : Started CitaBellaApiApplication in 12.432 seconds (process r
citabella-backend | =====
citabella-backend | CARGANDO DEMO DATA...
citabella-backend | =====
citabella-backend | DEMO DATA CARGADA
citabella-backend | =====
citabella-backend | 2026-05-24T22:28:09.825Z INFO 1 --- [CitaBellaAPI] [nio-8080-exec-1] o.s.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
citabella-backend | 2026-05-24T22:28:09.825Z INFO 1 --- [CitaBellaAPI] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
citabella-backend | 2026-05-24T22:28:09.827Z INFO 1 --- [CitaBellaAPI] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
citabella-nginx | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
citabella-nginx | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
citabella-nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
citabella-nginx | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
citabella-nginx | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
citabella-nginx | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
citabella-nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
citabella-nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
citabella-nginx | /docker-entrypoint.sh: Configuration complete; ready for start up

```

Ilustración 24 docker compose logs

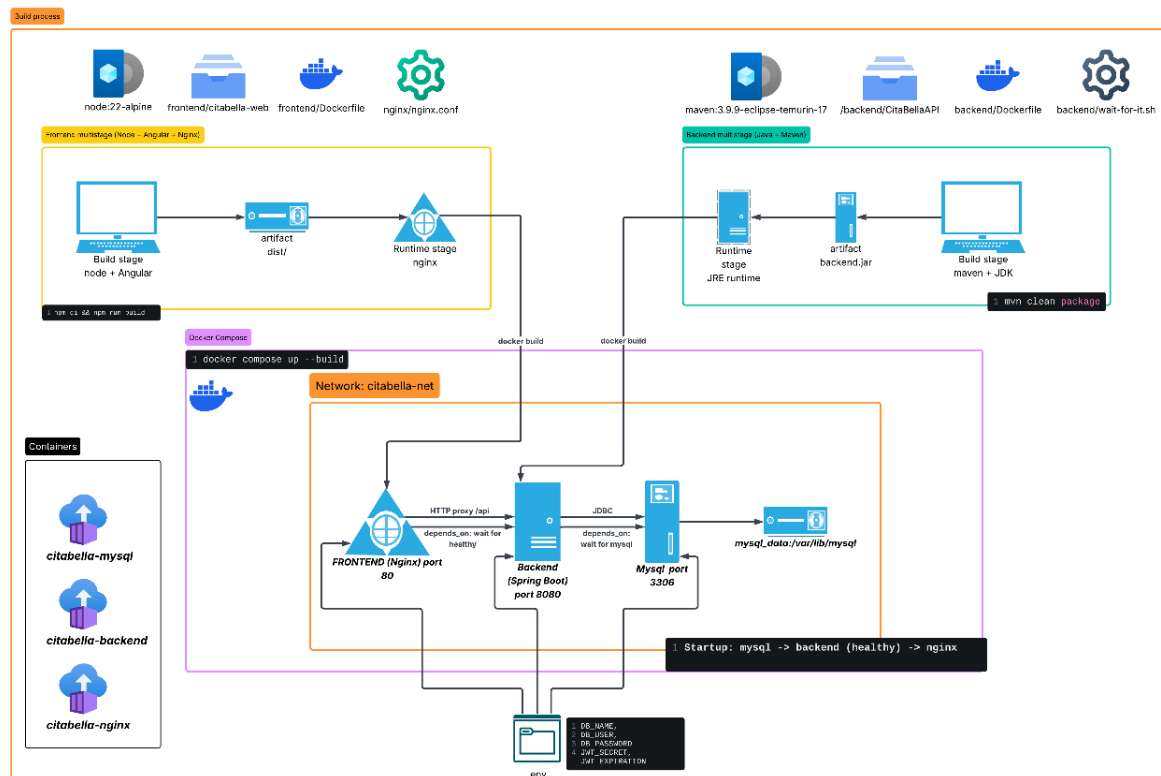


Ilustración 25 Diagrama Docker Compose

4.2.8 Justificación de decisiones técnicas

Las decisiones tecnológicas se tomaron priorizando tres criterios: adecuación al caso de uso, madurez del ecosistema y aprovechamiento de los conocimientos del ciclo.

- Spring Boot frente a Node.js o Quarkus:** Spring Boot es el estándar industrial para APIs REST en Java, con ecosistema maduro que integra Spring Security, Spring Data y validaciones sin configuración adicional. Su autoconfiguración permite arrancar rápidamente sin sacrificar control sobre los componentes. La alineación con Java como lenguaje principal del ciclo fue otro factor determinante.

- **JWT stateless frente a sesiones HTTP:** El sistema cuenta con dos clientes distintos: navegador web y app Flutter. Los JWT permiten autenticar ambos sin depender de sesiones HTTP basadas en cookies, facilitando además el escalado del sistema.
- **Angular frente a React o Vue:** para una aplicación de gestión con múltiples roles y vistas protegidas, Angular presenta ventajas estructurales: guardas de ruta declarativas (CanActivate), interceptores HTTP globales para añadir el token JWT desde un único punto, inyección de dependencias con servicios singleton, y lazy loading nativo por módulos. React y Vue trasladan al desarrollador la elección y coordinación de estas soluciones, añadiendo complejidad en un proyecto individual.
- **Flutter con InAppWebView frente a app nativa completa:** desarrollar una app Android nativa habría requerido duplicar íntegramente la lógica de negocio, la interfaz, las validaciones y la autenticación JWT ya implementadas en Angular. La estrategia del contenedor WebView reutiliza el 100% del frontend web, añadiendo únicamente la capa nativa (splash, conectividad, navegación) con un esfuerzo de desarrollo adicional mínimo.
- **MySQL frente a PostgreSQL:** ambos son sistemas relacionales maduros compatibles con JPA/Hibernate. La elección de MySQL responde a la familiaridad previa del desarrollador y el soporte nativo en Spring Data JPA sin configuración adicional del dialecto.
- **Docker Compose frente a Kubernetes o despliegue manual:** Docker Compose es la solución de orquestación adecuada para un sistema de tres contenedores en un entorno académico o de pequeño despliegue. Kubernetes añadiría complejidad operativa no justificada para el alcance del proyecto. El despliegue manual eliminaría la reproducibilidad del entorno.

Las versiones utilizadas corresponden a las estables disponibles en el momento del inicio del desarrollo (septiembre 2025). Angular 21 fue lanzado en noviembre de 2025. La migración desde Angular 20 se realizó en siguiendo la guía oficial de actualización.

5. IMPLANTACIÓN

5.1 Backend — Spring Boot

5.1.1 Configuración del entorno

Tecnología	Versión	Rol
Java	17	Plataforma de ejecución del backend
Spring Boot	3.5.7	Framework backend REST
Maven	3.9.9	Gestión de dependencias y construcción
MySQL	8	Sistema gestor de base de datos
IntelliJ IDEA Ultimate	—	IDE de desarrollo

El fichero *application.properties* configura la conexión a la base de datos, la clave JWT y el tiempo de expiración del token.

En el entorno Docker, toda esta configuración se sobrescribe mediante variables de entorno del fichero *.env*, activando el perfil docker que carga *application-docker.properties*.

Esta separación de perfiles garantiza que la configuración de desarrollo nunca se desplegará accidentalmente en producción.

5.1.2 Docker Compose: Servicios levantados

La configuración de Docker Compose define tres servicios que arrancan en orden de dependencia controlado por healthchecks:

1. **MySQL** arranca primero. Su healthcheck verifica disponibilidad mediante mysqladmin ping.
2. **Backend** espera a MySQL con condition: service_healthy y adicionalmente usa wait-for-it.sh para confirmar el puerto TCP. Una vez listo, expone healthcheck en /actuator/health con interval: *10s*, timeout: *5s*, retries: *10*, start_period: *40s*.
3. **Nginx** espera al backend con condition: service_healthy antes de iniciar.

Nginx sirve los archivos estáticos de Angular con **try_files \$uri \$uri/ /index.html** y redirige /api/ al backend mediante **proxy_pass**. La directiva *set \$upstream* junto con el resolver DNS interno de Docker (127.0.0.11) obliga a Nginx a resolver la IP del backend en tiempo de ejecución, evitando fallos si el backend tarda en inicializarse.

DOCUMENTACIÓN TÉCNICA COMPLETA DEL DESPLIEGUE EN:

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA/blob/main/docs/DevOps.md

5.1.3 Spring Boot: Estructura, paquetes, endpoints CRUD

El proyecto backend se organiza bajo el paquete base `com.citabella.citabellaapi`:

Paquete	Responsabilidad
config/	Seguridad (SecurityConfig, JwtUtil, JwtAuthFilter), CORS, Swagger, DemoDataInitializer
entity/	Entidades JPA organizadas en subpaquetes por dominio
dto/	Records Java inmutables con validaciones Jakarta, organizados por dominio
repository/	Interfaces JpaRepository con consultas JPQL personalizadas
service/	Interfaces e implementaciones separadas; lógica de negocio completa
controller/	Endpoints REST sin lógica de negocio; delegan en servicios
mapper/	Clases estáticas de conversión Entity ↔ DTO
exception/	GlobalExceptionHandler con respuestas JSON estructuradas

La API expone 46 endpoints documentados en Swagger UI. El inventario completo se incluye en la sección 4.2.4.

[VER SECCIÓN 4.2.4 — TABLA COMPLETA DE ENDPOINTS]

5.1.4 Seguridad: JWT, roles

El sistema implementa autenticación **stateless** mediante **JSON Web Tokens con algoritmo HS256**. El flujo completo:

1. El usuario envía credenciales a `/api/auth/login`.
2. El backend valida la contraseña contra el hash BCrypt almacenado.
3. Si son correctas, se genera un JWT firmado con HS256 que incluye username y rol. El token expira en **24 horas** (configurable mediante `JWT_EXPIRATION`, por defecto 86400000 ms).
4. El cliente almacena el token en `localStorage` (web) o en el almacenamiento del `InAppWebView` (móvil).
5. El `AuthInterceptor` de Angular añade automáticamente `Authorization: Bearer <token>` a todas las peticiones.
6. El `JwtAuthFilter` (extiende `OncePerRequestFilter`) intercepta cada petición, valida el token con `JwtUtil`, y si es válido inyecta el usuario y su rol en el `SecurityContextHolder`.

Los roles del sistema son **ADMIN**, **EMPLOYEE**, **CLIENT** y **USER**. La protección de rutas se aplica en dos niveles: rutas completamente públicas en `SecurityConfig`, y control granular por método con `@PreAuthorize`.

Las contraseñas se almacenan como hash BCrypt (factor de coste por defecto: 10), nunca en texto plano. La protección CSRF está desactivada por diseño en arquitecturas stateless.

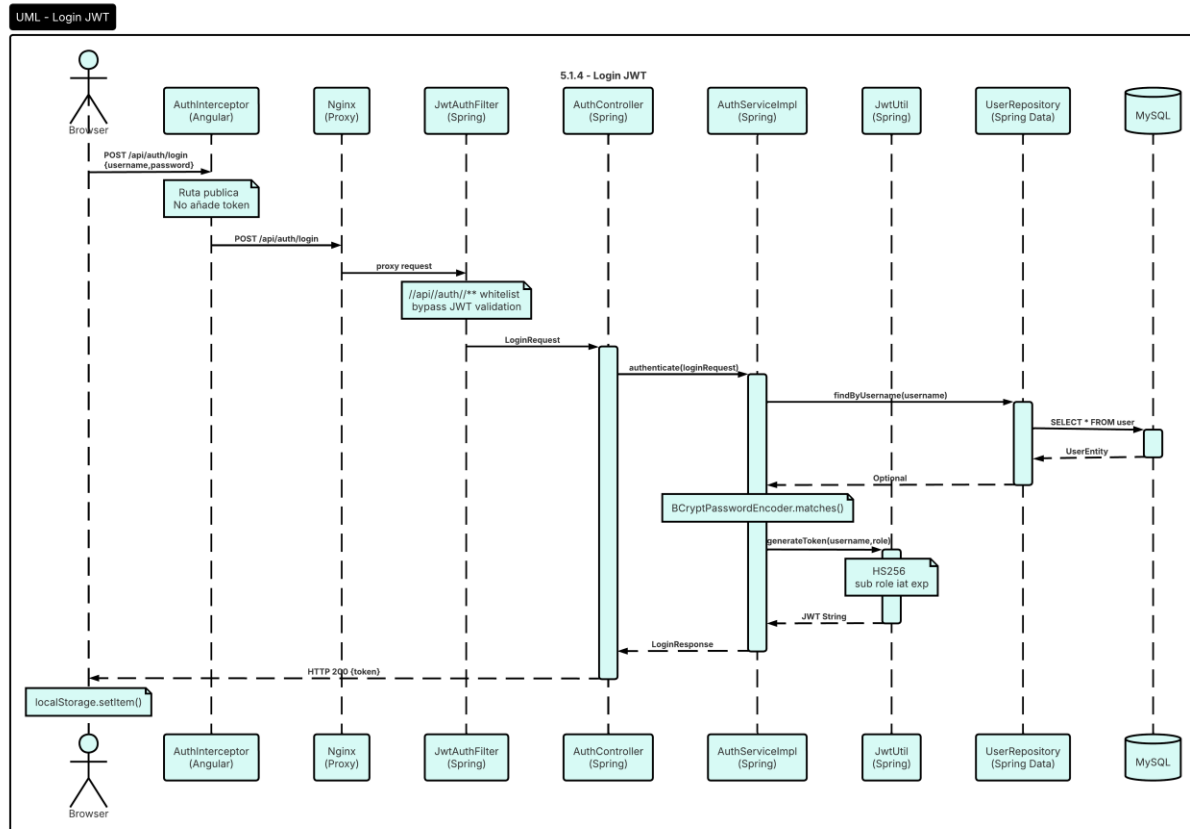


Ilustración 26 Diagrama UML Login JWT

Flujo de Autenticación JWT

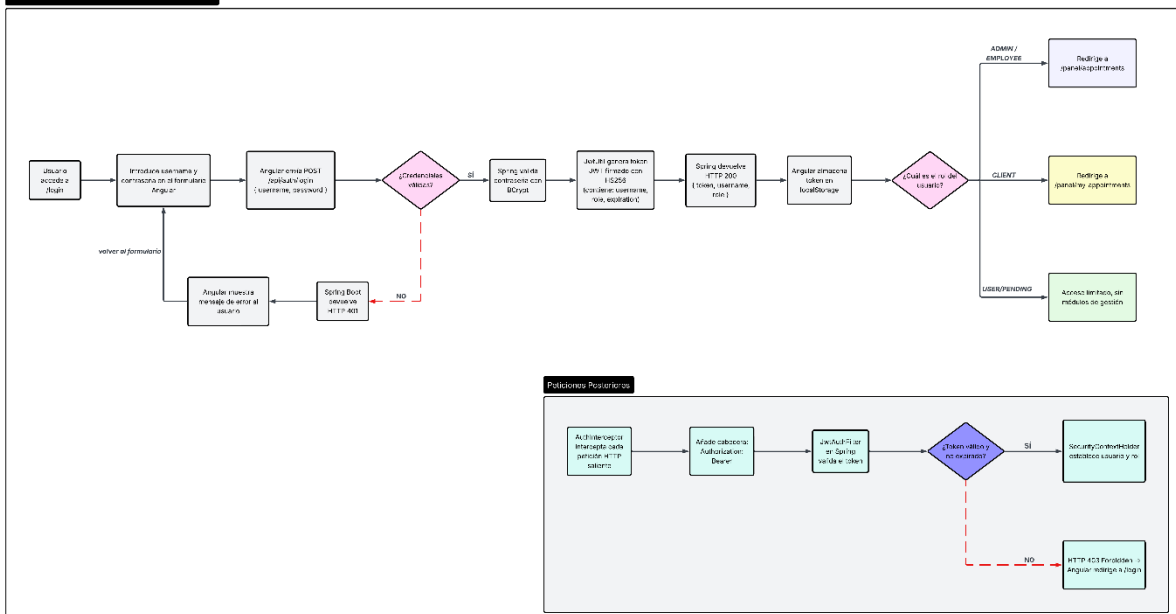


Ilustración 27 Flujo de Autenticación JWT

5.1.5 Validaciones de negocio

Las validaciones se aplican en dos niveles:

Nivel 1 — DTOs:

Anotaciones Jakarta Validation

(@NotBlank, @NotNull, @Email, @Size, @DecimalMin, @Min, @Positive)

Devolviendo error 400 estructurado si falla alguna validación.

Nivel 2 — Servicios:

Unicidad de variables, verificación de usuario sin perfil vinculado, y detección de solapamientos de citas mediante `AppointmentRepository.hasOverlap()`.

El campo `hasOverlap` no bloquea la creación; marca la cita con advertencia visual en el frontend.

Máquina de estados de citas:

PENDING → CONFIRMED | CANCELLED

CONFIRMED → IN_PROGRESS | CANCELLED | NO_SHOW

IN_PROGRESS → COMPLETED | CANCELLED

Borrado lógico: clientes, empleados, tratamientos y productos se desactivan con `active = false`. Los usuarios se bloquean con `accountStatus = LOCKED`. Ningún registro se elimina físicamente.

Todos los errores son capturados por `GlobalExceptionHandler`, que devuelve siempre una respuesta JSON estructurada sin exponer trazas internas.

5.1.6 Scripts SQL e inicialización

`DemoDataInitializer` se ejecuta al arrancar cuando `app.demo.enabled=true` y la base de datos está vacía (`userRepository.count() == 0`). Esta verificación garantiza **idempotencia**.

Entidad	Cantidad creada
Roles (ADMIN, EMPLOYEE, CLIENT, USER)	4
Administrador del sistema	1
Empleados de demostración	10
Clientes de demostración	45
Usuarios adicionales de demostración	6
Tratamientos de demostración	16
Productos de demostración	20

Credenciales del administrador de demostración en desarrollo:

usuario *citabella*, contraseña *citabella123*.

5.1.7 Logs y auditoría

El sistema dispone de una entidad AuditLog en la base de datos diseñada para registrar acciones relevantes (INSERT, UPDATE, DELETE) con el usuario que las realizó, la entidad afectada y la marca de tiempo.

La implementación activa de este módulo está planificada para la Fase 2. En la Fase 1, los logs estándar de Spring Boot registran las peticiones entrantes, los errores no controlados y la actividad del DemoDataInitializer al arranque.

5.2 Frontend — Angular 21

5.2.1 Estructura y arquitectura de módulos

Módulo	Ruta base	Componentes principales	Roles
PublicModule	/	Home, ServicesPage, ProductsPage	Todos
AuthModule	/login	Login	Todos
RegisterModule	/register	Register	Todos
AppointmentsModule	/panel/appointments	AppointmentCalendar, AppointmentForm, AppointmentModal	ADMIN, EMPLOYEE
ClientsModule	/panel/clients	ClientList, ClientForm	ADMIN, EMPLOYEE
ProductModule	/panel/products	ProductList, ProductForm	ADMIN, EMPLOYEE
EmployeesModule	/panel/employees	EmployeeList, EmployeeForm	ADMIN
TreatmentsModule	/panel/treatments	TreatmentList, TreatmentForm	ADMIN
AdminModule	/panel/admin	UserList, UserForm	ADMIN
MyAppointmentsModule	/panel/my-appointments	MyAppointmentsCalendar	CLIENT

Servicios principales:

AuthService, AppointmentService, ClientService, EmployeeService, TreatmentService, ProductService, UserService — cada uno encapsula la comunicación HTTP con el backend y devuelve Observable<T> de RxJS.

Servicios de infraestructura:

- ThemeService (tema oscuro/claro/auto con Signals)
- ToastService (notificaciones emergentes, 4 segundos)
- ConfirmService (diálogo de confirmación reutilizable, Promise<boolean>)
- BreadcrumbService (navegación reactiva con BehaviorSubject)

5.2.2 Guardas e interceptor JWT

AuthGuard:

Verifica la existencia de token en localStorage. Sin token → redirige a /login.

RoleGuard:

Compara AuthService.getRole() con los roles permitidos en data.roles.

Rol insuficiente → redirige a /unauthorized.

AuthInterceptor:

Interceptor HTTP global registrado en CoreModule. Añade Authorization: Bearer <token> a todas las peticiones salientes.

Gestiona errores 401 (limpia sesión → /login) y 403 (mantiene sesión → /) de forma centralizada. Ningún servicio de dominio necesita gestionar cabeceras ni errores de autorización manualmente.

5.2.3 Calendario de citas con FullCalendar

Módulo de citas FullCalendar 6.1.20:

Tres vistas (dayGridMonth, timeGridWeek, timeGridDay). Las citas se representan con código de colores según estado e indicador visual para solapamientos.

La creación de citas se realiza mediante un **asistente de 3 pasos** (Paso 1: cliente y horario → Paso 2: tratamientos y empleado → Paso 3: confirmación) o mediante el formulario completo alternativo.

El modal AppointmentModal gestiona el ciclo de vida posterior mostrando únicamente las transiciones válidas según el estado actual de la cita.

5.2.4 Sistema de temas oscuro/claro con Angular Signals

ThemeService implementa tres modos:

Light (Navy — #0A0F1E)

Dark (Gold — #C8A96E)

Auto (sigue la preferencia del sistema operativo mediante matchMedia).

El estado se gestiona con un **Signal reactivo** (isDark) que actualiza la clase del body y persiste la preferencia en localStorage bajo la clave citabella-theme.

El sistema visual se basa en **más de 60 variables CSS** definidas en styles/tokens.css (colores, sombras, bordes, radios, gradientes, breakpoints). Cambiar el tema completo se reduce a modificar estos valores, garantizando coherencia y facilitando el mantenimiento.

DOCUMENTACIÓN TÉCNICA COMPLETA DEL FRONTEND EN:

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA/blob/main/docs/Frontend.md

5.2.6 Métricas de rendimiento en producción

La build de producción generada por Angular CLI (con minificación, tree-shaking y hashing de assets) fue auditada en Desktop con Google Lighthouse sobre la build servida por Nginx. Los resultados sobre la ruta pública del sistema muestran:

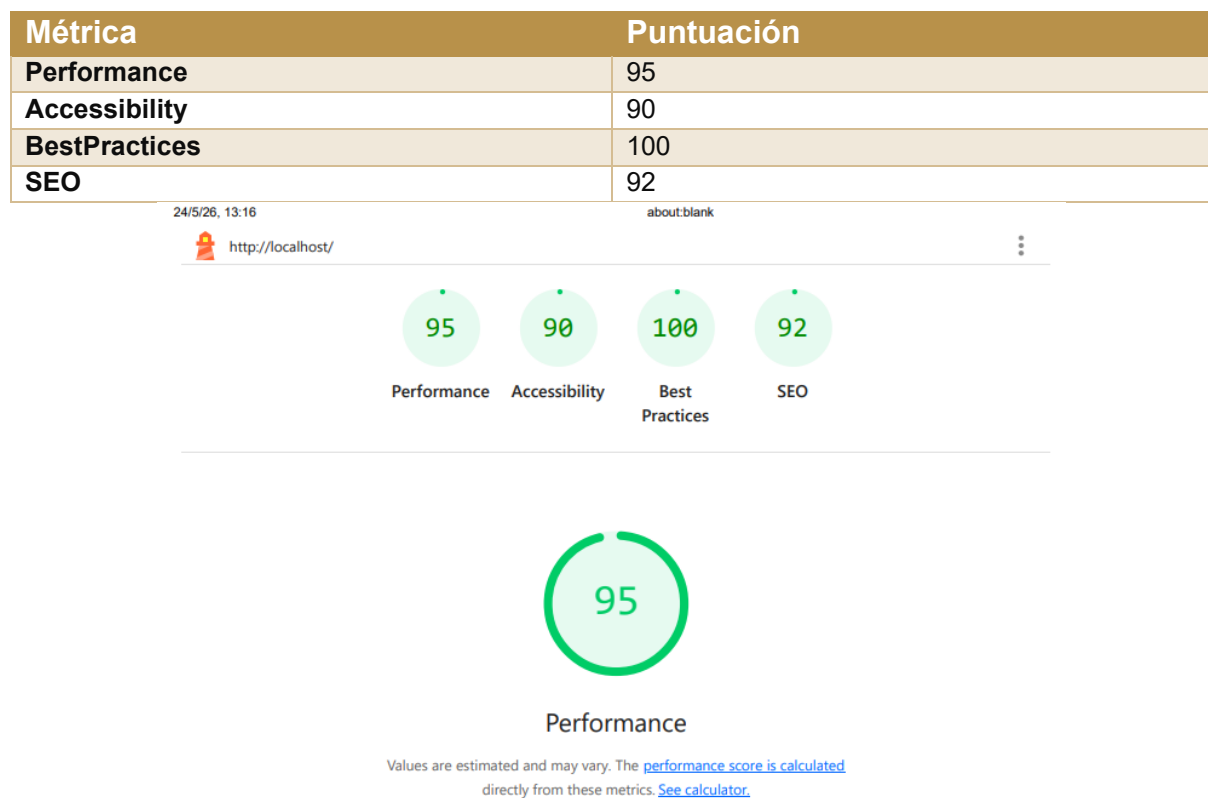


Ilustración 28 Métrica de rendimiento Google Lighthouse

5.3 Aplicación móvil — Flutter

5.3.1 Arquitectura del contenedor WebView

La app actúa como un **shell nativo Flutter** con flutter_inappwebview ^6.1.5 que envuelve la SPA Angular. Se eligió flutter_inappwebview en lugar de webview_flutter por ofrecer control avanzado: soporte completo de JavaScript, DOM Storage e IndexedDB (necesario para JWT en localStorage), barra de progreso real, pull-to-refresh y depuración remota con Chrome DevTools.

Componente	Tecnología	Versión
Framework	Flutter SDK	≥3.3.0
WebView	flutter_inappwebview	^6.1.5
Estado	provider	^6.1.2
Conectividad	connectivity_plus	^6.1.1
Splash nativo	flutter_native_splash	^2.4.3
Icono	flutter_launcher_icons	^0.14.4

5.3.2 Gestión de conectividad y estados del WebView

El WebViewStateController (patrón ChangeNotifier + Provider) gestiona cinco estados:

Estado	Descripción	UI mostrada
initializing	Verificando conectividad	LoadingOverlay
loading	InAppWebView cargando la SPA	LoadingOverlay con barra de progreso real (0–100%)
loaded	Angular lista y visible	WebView (overlay hace fade out)
offline	Sin red disponible	OfflineScreen — icono wifi_off
serverError	Red OK pero servidor no responde en 8 s	OfflineScreen — icono dns_outlined

El ConnectivityService distingue sin red versus servidor inalcanzable, mostrando mensajes de error precisos al usuario.

5.3.3 Funcionalidades nativas

Splash: generado con flutter_native_splash, compatible con Android 12+ Splash Screen API. Dos variantes: fondo #E4F0F6 con logo claro (light) y fondo #0D0D0D con logo oscuro (dark).

Tema Material 3: la app se adapta al tema del sistema operativo. La paleta está sincronizada con los design tokens del frontend Angular.

Botón Atrás de Android: gestionado con PopScope. Si hay historial en el WebView → navega atrás en Angular. Si no hay historial → diálogo de confirmación de salida → `SystemNavigator.pop()`.

Pull-to-refresh: implementado con `PullToRefreshController`; se desactiva al completar la carga.

Android: `INTERNET`, `ACCESS_NETWORK_STATE`, `ACCESS_WIFI_STATE`. uses `CleartextTraffic="true"` para desarrollo HTTP (desactivar en producción con HTTPS).

DOCUMENTACIÓN TÉCNICA COMPLETA EN:

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA/blob/main/docs/App-Android-IOS.md

5.4 DevOps y despliegue con Docker

5.4.1 Dockerfiles multietapa

Backend (Java 17):

- Etapa 1 (compilación): `maven:3.9.9-eclipse-temurin-17` descarga dependencias y genera el JAR con `mvn clean package -DskipTests`.
- Etapa 2 (ejecución): `eclipse-temurin:17-jdk-jammy` copia solo el JAR, instala `netcat-openbsd` y `curl`, y usa `wait-for-it.sh mysql:3306 -- java -jar app.jar` como punto de entrada.

Frontend (Node 22 + Nginx):

- Etapa 1 (compilación): `node:22-alpine` — instala dependencias npm y ejecuta `npm run build --configuration production` (minificación, tree shaking, hashing de assets).
- Etapa 2 (servidor): `nginx:alpine` — copia solo los archivos estáticos del directorio `dist/*/browser`.

5.4.2 Variables de entorno y persistencia

Toda la configuración sensible (credenciales de BD, clave JWT, datos del administrador inicial) se centraliza en el fichero `.env` excluido del control de versiones. Se incluye `.env.example` como plantilla.

Los datos de MySQL persisten en el volumen Docker nombrado `mysql_data`. El volumen solo se elimina explícitamente con `docker compose down -v`. Backend y Nginx son completamente stateless.

5.4.3 Comandos de operación

Comando	Efecto
docker compose up -d	Levanta el entorno completo. Primera ejecución: compila el código.
docker compose up -d --build	Recompila y levanta. Necesario tras cambios en el código.
docker compose down	Detiene contenedores. Los datos de MySQL persisten.
docker compose down -v	Elimina contenedores y el volumen de datos. Irreversible.

Tras el arranque: aplicación disponible en <http://localhost> y Swagger UI en <http://localhost/swagger-ui.html>.

DOCUMENTACIÓN TÉCNICA COMPLETA EN:

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA/blob/main/docs/DevOps.md

5.5 Gestión del código con Git y GitHub

5.5.1 Estrategia de ramas

El repositorio sigue un modelo de ramificación simplificado basado en Git Flow, adaptado a un proyecto individual:

Rama	Propósito
main	Rama principal estable. Recibe merges solo cuando una funcionalidad está completa
develop	Rama de integración. Base para el desarrollo de nuevas funcionalidades
feature/backend	Desarrollo del backend Spring Boot
feature/frontend	Desarrollo del frontend Angular

5.5.2 Convención de commits

Los commits siguen el estándar **Conventional Commits**:

Prefijo	Uso
feat:	Nueva funcionalidad
fix:	Corrección de error
docs:	Cambios en documentación
refactor:	Reorganización sin cambio de comportamiento
chore:	Tareas de mantenimiento

Nota sobre identidad de commits

Durante el desarrollo, algunos commits registrados en el repositorio aparecen bajo el identificador de autor shdw en lugar del nombre oficial del proyecto. Esto se debe a diferencias en la configuración local de Git entre los

distintos entornos de trabajo utilizados (equipo principal con Windows 11 y entorno WSL2), donde la variable `user.name` no estaba sincronizada entre sesiones. En todos los casos, la autenticación con GitHub se realizó mediante el mismo token personal, por lo que todos los commits pertenecen inequívocamente al mismo autor. Para verificarlo, el historial de contribuciones del repositorio refleja un único colaborador activo en la pestaña Contributors de GitHub Insights.

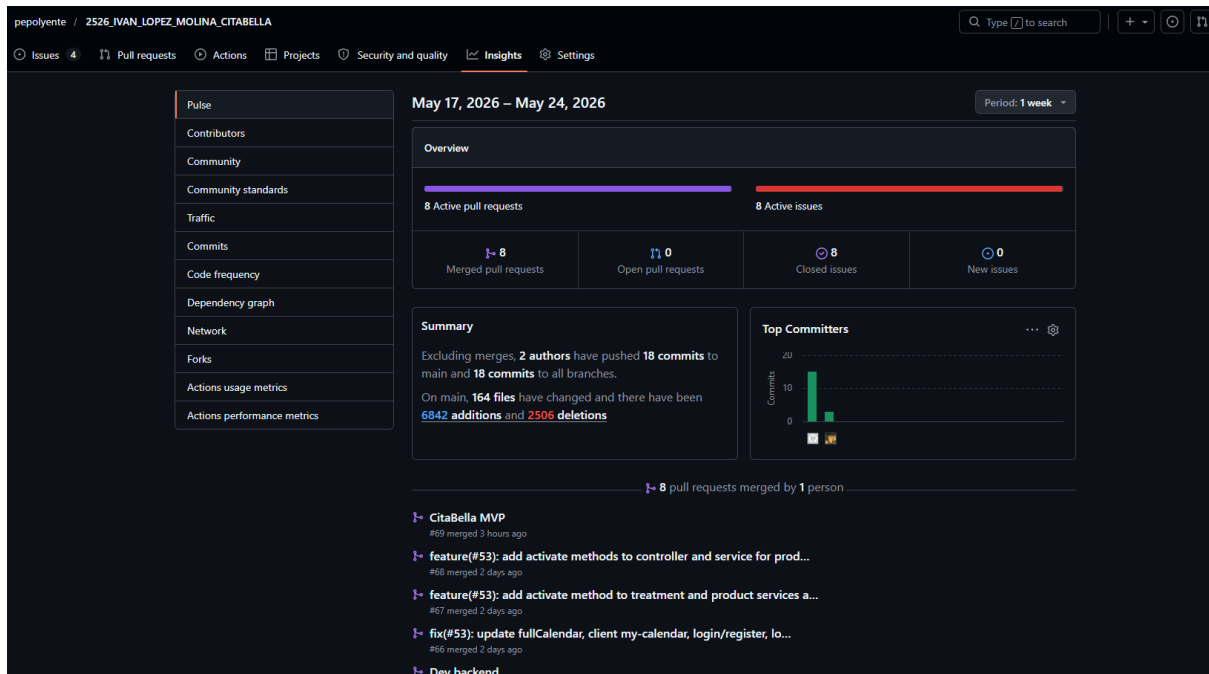


Ilustración 29 Git Hub Pulls Insights

5.5.3 GitHub Projects y seguimiento de tareas

El seguimiento del trabajo se realizó mediante **GitHub Projects** con tablero Kanban organizado en columnas *To Do*, *In Progress* y *Done*. Las tareas más relevantes se asociaron a *issues* y se agruparon en *milestones* por fase del proyecto.

5.6 Pruebas del sistema

5.6.1 Pruebas de la API REST

Se han desarrollado 67 requests organizadas en colecciones Bruno que cubren todos los endpoints de la API. Las pruebas validan los flujos principales: autenticación correcta e incorrecta, control de acceso por rol, ciclo de vida completo de citas, y manejo de errores. La colección completa está disponible en `docs/CitaBella-Bruno-Tests.zip`.

5.6.2 Pruebas de integración manual

Se han validado las 20 historias de usuario del MVP siguiendo los criterios de aceptación Gherkin definidos. Los escenarios de solapamiento de citas y transiciones de estado han sido específicamente verificados.

5.6.3 Pruebas de la aplicación móvil

La app Flutter fue probada en dispositivo real (Realme C55, Android 15.0). Se verificaron los cinco estados del WebViewStateController: initializing, loading, loaded, offline y serverError.

5.6.4 Trabajo futuro en pruebas

Los tests unitarios con JUnit 5 para AppointmentService y AuthService están identificados como mejora prioritaria para la Fase 2, dado el impacto de estas clases en la lógica de negocio crítica.

6. RECURSOS

6.1 Hardware utilizado

Dispositivo	Uso	Especificaciones
Ordenador de desarrollo	Desarrollo, compilación y pruebas	Windows 11, x64, 16 GB RAM, SSD
Dispositivo móvil Android	Pruebas de la app Flutter y visualización responsive	Realme C55, 15.0

6.2 Software y herramientas

Software	Versión	Uso
Java	17	Plataforma de ejecución del backend
Spring Boot	3.5.7	Framework backend REST
Maven	3.9.9	Gestión de dependencias y build del backend
MySQL	8	Sistema gestor de base de datos relacional
Angular	21	Framework frontend SPA
TypeScript	5.9	Lenguaje del frontend
RxJS	7.8	Programación reactiva en Angular
FullCalendar	6.1.20	Componente de calendario interactivo
Flatpickr	4.6.13	Selector de fecha y hora
Node.js	22	Entorno de construcción del frontend
Flutter SDK	≥3.3.0	Desarrollo de la aplicación móvil Android
flutter_inappwebview	^6.1.5	WebView avanzado para Flutter
connectivity_plus	^6.1.1	Detección de conectividad en Flutter
flutter_native_splash	^2.4.3	Splash nativo para Flutter
Docker Desktop	—	Contenerización y orquestación local
Docker Compose	v2	Orquestación de los tres servicios
Nginx	Alpine	Servidor web y proxy inverso
IntelliJ IDEA Ultimate	—	IDE para backend Java y frontend Angular
Git	—	Control de versiones distribuido
GitHub	—	Repositorio remoto, Projects y seguimiento
Bruno	—	Pruebas manuales de la API REST
Swagger UI (Springdoc)	—	Documentación interactiva de la API

6.3 Sistemas operativos

Sistema	Uso
Windows 11	Sistema operativo principal de desarrollo
Linux — Ubuntu (WSL2)	Ejecución de Docker y comandos de terminal Unix
Android	Pruebas de la aplicación móvil Flutter

6.4 Repositorio y servicios externos

Servicio	Detalle
Repositorio GitHub (público)	https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA
Documentación API estática	docs/api-docs.html (explorable sin levantar el servidor)
Imágenes Docker base	mysql:8, maven:3.9.9-eclipse-temurin-17, eclipse-temurin:17-jdk-jammy, nginx:alpine, node:22-alpine

7. CONCLUSIONES

7.1 Grado de consecución de objetivos

#	Objetivo	Estado	Observaciones
1	Modelo de datos relacional normalizado para las tres fases	Completado	Incluye tablas de ventas, stock y notificaciones preparadas para Fases 2 y 3
2	API REST con seguridad JWT y control por roles	Completado	46 endpoints documentados en Swagger Roles ADMIN, EMPLOYEE, CLIENT implementados
3	Frontend Angular con lazy loading, interceptores y guardas	Completado	10 módulos con lazy loading AuthInterceptor y RoleGuard implementados
4	Aplicación móvil Android con Flutter	Completado	InAppWebView con splash nativo, gestión de conectividad y navegación
5	Despliegue con Docker Compose (un único comando)	Completado	Tres contenedores orquestados con healthchecks y dependencias encadenadas
6	Buenas prácticas: capas, DTOs, validaciones, manejo de errores	Completado	Arquitectura por capas Records Java para DTOs GlobalExceptionHandler
7	Trazabilidad con Git y GitHub Projects	Completado	Repositorio público con commits descriptivos y GitHub Projects

Los objetivos de fases posteriores — gestión completa de ventas e inventario (Fase 2) y notificaciones automáticas (Fase 3) — están planificados y sus entidades ya modeladas, pero fuera del alcance del MVP entregado.

7.2 Problemas encontrados y soluciones

Problema 1: Curva de aprendizaje de Spring Security con JWT stateless

La integración de Spring Security con autenticación JWT stateless requiere comprender en profundidad la cadena de filtros de seguridad.

La documentación oficial cambió significativamente en Spring Security 6 (eliminando *WebSecurityConfigurerAdapter*), lo que generó confusión con los recursos educativos disponibles. Se resolvió mediante estudio de la documentación oficial de Spring Security 6 y recursos de *Baeldung*, implementando progresivamente: primero securización básica, luego el filtro JWT personalizado, y finalmente configuración granular por endpoint.

Problema 2: Detección de solapamientos de citas

La consulta JPQL para detectar si una nueva cita se solapa con alguna existente del mismo empleado requirió análisis cuidadoso de la lógica de intervalos temporales. La condición inicial estaba incorrectamente formulada y no detectaba todos los casos posibles. Se resolvió usando el principio del complementario: dos intervalos se solapan si y solo si no es cierto que uno termina antes de que el otro empiece. La condición final verifica *nuevaInicio < existenteFin AND nuevaFin > existenteInicio*.

Problema 3: Configuración de CORS y proxy Angular

Durante el desarrollo, el frontend (puerto 4200) y el backend (puerto 8080) requerían ajuste coordinado de CORS y del proxy de Angular para que las peticiones a /api se redirigieran correctamente en ambos entornos (desarrollo con proxy Angular y producción con Nginx). La solución fue configurar el proxy Angular para redirigir /api a localhost:8080 en desarrollo, y Nginx para hacer lo mismo en producción, con la configuración CORS del backend restringida a los orígenes esperados.

Problema 4: Sistema de temas con Angular Signals en componentes NgModule

La integración del ThemeService reactivo basado en Signals con componentes organizados en NgModule (no standalone) requirió entender las diferencias en el ciclo de vida entre ambos modelos. Los Signals no se actualizaban correctamente en la vista hasta corregir el patrón de consumo en el template (llamar al Signal como función *themeService.isDark()*, no como propiedad) y registrar el ThemeService correctamente como providedIn: 'root'.

Problema 5: Orden de arranque de contenedores Docker

Docker Compose con depends_on básico solo espera a que el contenedor esté en ejecución, no a que el servicio interno esté disponible. El backend intentaba conectarse a MySQL antes de que la base de datos estuviera lista para aceptar conexiones. Se resolvió configurando un healthcheck en el contenedor MySQL (mysqladmin ping) y usando condition: service_healthy en el depends_on del

backend, garantizando que Spring Boot no intente la conexión hasta que MySQL esté completamente listo.

7.3 Lecciones aprendidas

El desarrollo de CitaBella ha consolidado los conocimientos del ciclo y añadido un conjunto de competencias técnicas y metodológicas de valor profesional: arquitectura desacoplada, autenticación stateless con JWT, contenerización con Docker, gestión de estado reactivo con Signals y especificación formal de requisitos con historias de usuario.

Tres lecciones concretas destacan sobre las demás:

Spring Security 6 requiere configuración explícita. La migración de WebSecurityConfigurerAdapter a la API de SecurityFilterChain no está bien documentada en los recursos educativos disponibles. El CSRF disable para arquitecturas stateless, la configuración de CORS y la cadena de filtros JWT deben declararse explícitamente; ningún ejemplo básico de la documentación oficial lo hace evidente.

La detección de solapamientos requiere lógica de complementario. La condición JPQL inicial era incorrecta porque intentaba detectar el solapamiento directamente. La solución correcta es el principio del complementario: dos intervalos se solapan si y solo si $(nuevaFin \leq existenteInicio \vee nuevaInicio \geq existenteFin)$, lo que simplifica la consulta y cubre todos los casos.

Docker depends_on no garantiza disponibilidad del servicio. depends_on solo espera a que el contenedor arranque, no a que el servicio interno esté listo. El backend intentaba conectarse a MySQL antes de que la base de datos aceptara conexiones JDBC. La solución requirió combinar healthcheck en MySQL con condition: service_healthy y wait-for-it.sh para confirmar el puerto TCP.

La experiencia de trabajar sobre un caso real con una usuaria final ha puesto en valor la importancia del análisis de necesidades y la comunicación con el cliente.

7.4 Mejoras futuras y roadmap

Fase 2 — Inventario y ventas:

- Registro de ventas asociadas a citas con detalle de tratamientos y productos al precio vigente.
- Control de stock con movimientos INBOUND/OUTBOUND/ADJUSTMENT y alertas visuales para productos críticos.

- Informes de ingresos por período con KPIs: total de ingresos, citas completadas, top tratamientos.

Fase 3 — Notificaciones automáticas:

- Recordatorios de cita 24 horas antes por email para citas en estado CONFIRMED o PENDING.
- Confirmación automática por email al pasar una cita de PENDING a CONFIRMED.
- Notificación de cancelación al cliente si la canceló el negocio (no el propio cliente).
- Felicitación de cumpleaños automática para clientes con fecha de nacimiento registrada.

Mejoras transversales:

- Dashboard con métricas del negocio: citas por período, empleado más activo, clientes más frecuentes.
- Auditoría completa de acciones mediante la entidad AuditLog ya modelada.
- Implementación de HTTPS en producción y restricción del CORS a orígenes específicos.
- Tests unitarios con JUnit para los servicios de negocio más críticos (AppointmentService, AuthService).
- Componentes Flutter nativos para pantallas críticas como la agenda diaria y la creación rápida de citas.

7.5 Cumplimiento de Resultados de Aprendizaje

La siguiente tabla relaciona los Resultados de Aprendizaje de los módulos del ciclo con las evidencias concretas del proyecto:

Módulo	RA clave	Evidencia en CitaBella
Sistemas Informáticos	RA5, RA6	Red Docker app-net Proxy inverso Nginx JWT CORS Roles
Bases de Datos	RA2, RA3, RA6	Esquema en 3FN Consultas JPQL Diagrama ER completo
Programación	RA4, RA6, RA7, RA9	Arquitectura por capas Records Java Interfaces ServiceImpl @Transactional

Lenguajes de Marcas	RA3, RA4, RA5, RA6	TypeScript + Angular Jakarta Validation Mappers Entity ↔ DTO API 100% JSON
Entornos de Desarrollo	RA3, RA5, RA6	Pruebas manuales Bruno Diagrama de clases UML Diagramas de secuencia JWT
Acceso a Datos	RA2, RA3, RA6	Spring Data JPA; Hibernate ORM Repositorios inyectables
Desarrollo de Interfaces	RA3, RA4, RA6, RA7, RA8	Componentes reutilizables Modo oscuro/claro Swagger UI Build Docker Pruebas HU
Multimedia y Móviles	RA1, RA2	Flutter SDK Flutter_inappwebview Connectivity_plus Flutter_native_splash
Servicios y Procesos	RA1, RA4, RA5	Docker Compose (múltiples procesos) API REST stateless paginada JWT + BCrypt
ERP/CRM	RA1, RA3, RA4	CRM ligero Gestión de citas/clientes/usuarios Personalización para "Ruth Molina"
Proyecto Final	RA1, RA2, RA3, RA4	Análisis, Diseño, implantación y documentación completos GitHub Projects

8. REFERENCIAS Y BIBLIOGRAFÍA

[1] Spring Boot Documentation.

<https://docs.spring.io/spring-boot/> (consulta: enero 2026)

[2] Spring Security Documentation.

<https://docs.spring.io/spring-security/> (consulta: enero 2026)

[3] Angular Documentation.

<https://angular.dev/> (consulta: marzo 2026)

[4] Angular Lazy Loading Modules.

<https://angular.dev/guide/ngmodules/lazy-loading> (consulta: abril 2026)

[5] Angular — HTTP Interceptors.

<https://angular.dev/guide/http/interceptors> (consulta: abril 2026)

[6] Angular Signals.

<https://angular.dev/guide/signals> (consulta: abril 2026)

[7] Flutter Documentation.

<https://docs.flutter.dev/> (consulta: enero 2026)

[8] flutter_inappwebview pub.dev.

https://pub.dev/packages/flutter_inappwebview (consulta: enero 2026)

[9] Docker Documentation.

<https://docs.docker.com/> (consulta: noviembre 2025)

[10] Docker Compose — Compose file reference.

<https://docs.docker.com/compose/compose-file/> (consulta: noviembre 2025)

[11] Nginx Reverse Proxy Module.

https://nginx.org/en/docs/http/nginx_http_proxy_module.html (consulta: abril 2026)

[12] Springdoc OpenAPI.

<https://springdoc.org/> (consulta: diciembre 2025)

[13] JSON Web Tokens — Introduction.

<https://jwt.io/introduction> (consulta: diciembre 2025)

[14] BCrypt — Spring Security Crypto.

<https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html> (consulta: diciembre 2025)

[15] Jakarta Persistence (JPA).

<https://jakarta.ee/specifications/persistence/> (consulta: abril 2026)

[16] MySQL 8.0 Reference Manual.

<https://dev.mysql.com/doc/refman/8.0/en/> (consulta: abril 2026)

[17] Maven — Official Documentation.

<https://maven.apache.org/guides/> (consulta: abril 2026)

[18] FullCalendar — Documentation.

<https://fullcalendar.io/docs> (consulta: marzo 2026)

[19] Flatpickr — Documentation.

<https://flatpickr.js.org/> (consulta: marzo 2026)

[20] Baeldung — Spring and Java Tutorials.

<https://www.baeldung.com/> (consulta: enero 2026)

[21] GitHub Docs — About branches.

<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches> (consulta: octubre 2025)

[22] GitHub Projects.

<https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects> (consulta: abril 2026)

[23] López Molina, Ivan. Repositorio del proyecto CitaBella [código fuente público].

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA (2025–2026)

9. ANEXOS

Anexo A — Entrevista con la propietaria del negocio

Fecha: Noviembre 2025

Participantes: Iván López Molina (entrevistador/desarrollador), Ruth Molina (propietaria de la peluquería)

Objetivo: Identificar los procesos actuales de gestión, las necesidades prioritarias del negocio y los requisitos funcionales del sistema CitaBella.

P: ¿Qué es CitaBella y cuál es su propósito principal?

R: *Es un sistema de gestión de citas y agenda para la peluquería. Lo que quiero es digitalizar la empresa y sus procesos: agrupar la información, hacer estadísticas y automatizar tareas que ahora se hacen a mano. No es solo para pedir citas, sino para tener todo el negocio organizado desde una sola herramienta.*

P: ¿Para quién está pensada la aplicación?

R: *Para dos tipos de usuarios: el cliente y el trabajador o dueño. Los clientes usarían principalmente la web para ver sus citas o pedir modificaciones, mientras que los trabajadores gestionarían todo desde una aplicación móvil o tablet, que es más cómoda para el día a día en el salón.*

P: ¿Cómo se gestionaría el registro de nuevos clientes?

R: *Prefiero que lo gestione el trabajador. Cuando se asigna una cita a un cliente nuevo, se le crea un usuario con nombre predefinido y contraseña generada, que se le entrega en tienda o por WhatsApp. El cliente entra con esas credenciales y puede cambiar su contraseña. No quiero que los clientes se registren solos de primeras, porque necesito verificar que son clientes reales del negocio.*

P: ¿Qué debe poder hacer un cliente desde la web?

R: *Revisar sus citas, ver el historial, pedir modificaciones o cancelaciones, y consultar información del negocio como servicios y productos destacados. También quiero que puedan guardar en su perfil información relevante como alergias o preferencias, para que los trabajadores la vean antes de atenderles.*

P: ¿Qué funciones necesita el trabajador en su herramienta?

R: *Crear citas, asignar clientes, confirmar o cancelar citas, y gestionar su propia agenda. También poder dar de alta clientes nuevos directamente desde la aplicación. La agenda tiene que ser clara: ver peticiones pendientes, confirmaciones y cancelaciones de un vistazo.*

P: ¿Qué papel tendría el rol de jefe o administrador?

R: *El jefe puede ver las agendas de todos los trabajadores, gestionar la información que aparece en la web del negocio y dar ciertos permisos a otros trabajadores. Pero solo el jefe puede borrar cuentas de trabajadores, eso no lo puede hacer nadie más.*

Aparte, habría un administrador técnico de CitaBella para cambios que requieran conocimientos del sistema.

P: ¿Qué páginas o secciones debe tener la web?

R: La página principal sería la del negocio, con el nombre, información del salón, servicios y productos. Desde ahí se puede acceder a la sección de citas eligiendo trabajador. Las secciones de agenda, perfil y gestión solo serían visibles tras iniciar sesión. También habría una página propia de CitaBella como plataforma.

P: ¿Qué sensación debería transmitir la web a quien la visita?

R: Profesionalidad y simplicidad. Que sea fácil de usar, especialmente para los clientes, porque no van a entrar muy a menudo. También tiene que servir un poco como escaparate del negocio, algo visual pero sin ser recargado.

P: ¿Qué funcionalidades se podrían dejar para una fase posterior?

R: Un panel de estadísticas o dashboard sería ideal, pero si no da tiempo no pasa nada. También cierto nivel de personalización del perfil de usuario podría ir en una segunda fase.

P: ¿Hay algo que definitivamente no quieres en la web?

R: De momento no tengo restricciones concretas de diseño. Lo más importante es que no sea confusa ni esté sobrecargada de opciones que los clientes no van a usar.

Los puntos clave extraídos de esta entrevista se recogen en las secciones 2.1 y 2.2 de esta memoria.

Anexo B — Reglas de negocio del sistema

Integridad y unicidad de datos

ID	Regla
RN-01	Un tratamiento no puede tener el mismo nombre que otro existente en el sistema
RN-02	Un producto no puede tener el mismo nombre que otro existente en el sistema
RN-03	Un usuario no puede registrarse con un email o nombre de usuario ya existentes
RN-04	El número de teléfono del cliente es único en el sistema
RN-05	El nombre del empleado es único en el sistema

Gestión del ciclo de vida de citas

ID	Regla
RN-06	Una cita debe tener al menos un tratamiento asociado
RN-07	Si una nueva cita se solapa con otra del mismo empleado, el sistema marca hasOverlap = true con advertencia visual. El solapamiento no bloquea la creación
RN-08	Los estados de una cita siguen el ciclo: PENDING → CONFIRMED → IN_PROGRESS → COMPLETED / CANCELLED / NO_SHOW. Las transiciones inversas no están permitidas

Gestión de usuarios y perfiles

ID	Regla
RN-09	Un usuario solo puede tener un perfil vinculado (cliente o empleado), no ambos simultáneamente
RN-10	Al vincular un usuario a un cliente, el rol cambia automáticamente a CLIENT y el estado a ACTIVE
RN-11	Al desvincular, el rol vuelve a USER y el estado a PENDING
RN-12	Al registrarse, un usuario recibe rol USER y estado PENDING hasta activación manual
RN-13	Las contraseñas se almacenan cifradas con BCrypt, nunca en texto plano

Borrado lógico y visibilidad

ID	Regla
RN-14	Productos y tratamientos se desactivan con active = false, preservando el historial
RN-15	Cientes y empleados usan borrado lógico con active = false
RN-16	Los usuarios se bloquean con accountStatus = LOCKED en lugar de eliminarse
RN-17	Un empleado desactivado no aparece como opción al crear nuevas citas

Visibilidad pública

ID	Regla
RN-18	Los productos de tipo INTERNAL no aparecen en la zona pública del sistema
RN-19	La vista pública de producto no expone precio de compra, proveedor ni flag de criticidad
RN-20	Se muestran como máximo 4 tratamientos y 4 productos destacados en la página principal

Anexo C — Índice de imágenes

Nota sobre resolución de imágenes: Las capturas y diagramas incluidos en este documento están optimizadas para visualización en pantalla e impresión en formato A4. La versión de alta resolución de todos los diagramas (ERD, UML, flujos, arquitectura Docker) y capturas del sistema está disponible y descargable directamente desde el repositorio público del proyecto:

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA/tree/main/docs/diagrams

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA/tree/main/docs/screenshots

El índice completo de ilustraciones incluidas en este documento es el siguiente:

Nº	Título	Sección
Ilustración 1	Priority Board GitHub Projects	3.2
Ilustración 2	GitHub Projects RoadMap	3.2
Ilustración 3	GitHub Commits over the last year	3.3
Ilustración 4	GitHub branches	3.3
Ilustración 5	Diagrama de contexto	4.1.1
Ilustración 6	Diagrama de flujo de citas	4.1.3
Ilustración 7	Diagrama ER base de datos	4.2.1
Ilustración 8	Diagrama Arquitectura del Sistema	4.2.3
Ilustración 9	Api Docs HTML Endpoints	4.2.4
Ilustración 10	Pantalla principal CitaBella Web	4.2.5
Ilustración 11	Pantalla Productos CitaBella Web	4.2.5
Ilustración 12	Pantalla Agenda de Citas — Vista Semanal	4.2.5
Ilustración 13	Vista Wizard creación de cita + flatpickr	4.2.5
Ilustración 14	Vista Wizard creación de cita — Paso 3	4.2.5
Ilustración 15	Pantalla Gestión Usuarios CitaBella Web	4.2.5
Ilustraciones 16–21	Flutter WebView + Splash + Conectividad	4.2.6
Ilustración 22	docker compose up -d	4.2.7
Ilustración 23	docker compose logs (arranque)	4.2.7
Ilustración 24	docker compose logs (backend listo)	4.2.7
Ilustración 25	Diagrama Docker Compose	4.2.7
Ilustración 26	Diagrama UML Login JWT	5.1.4
Ilustración 27	Flujo de Autenticación JWT	5.1.4
Ilustración 28	Métrica de rendimiento Google Lighthouse	5.2.6

Ilustración 29	GitHub Pulls Insights	5.5.2
Ilustración 30	Bruno Battery Tests API Endpoints	Anexo E
Ilustración 31	Prevención de riesgos — Leyenda	Anexo F
Ilustración 32	Riesgos Laborales	Anexo F

Anexo D — Historias de usuario

La especificación completa de las 28 historias de usuario (HU-01 a HU-28) está disponible en el repositorio del proyecto:

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA/blob/main/docs/Historias_de_usuario.md

A continuación, se incluye a modo de ejemplo la historia de usuario más representativa del sistema:

Leyenda de roles

- **Visitante** — cualquier persona sin autenticación
- **Cliente** — usuario registrado con perfil de cliente vinculado
- **Empleado** — trabajador con acceso operativo
- **Admin** — propietaria/administradora con acceso completo

HISTORIA DE USUARIO 6: FRENTE

ID: HU-06-CREATE-APPOINTMENT **TÍTULO:** Como empleado o administrador quiero crear una cita nueva asignando cliente, empleado, tratamientos y franja horaria para organizar la agenda del negocio.

REGLAS DE NEGOCIO:

- Una cita debe tener al menos un tratamiento asociado.
- Una cita debe tener un cliente, un empleado, una fecha de inicio y una fecha de fin.
- Al crearse, el estado inicial de la cita es PENDING.
- Si la franja horaria se solapa con otra cita del mismo empleado, el campo `hasOverlap` se marca como `true`. El sistema avisa, pero no bloquea la creación.
- Solo ADMIN y EMPLOYEE pueden crear citas.

DORSO

CRITERIOS DE ACEPTACIÓN:

Escenario 1: Creación exitosa de cita sin solapamiento Dado que el empleado "Ruth Molina" no tiene citas en el horario 10:00–11:00 del próximo lunes. Cuando un empleado crea una cita para el cliente "Ivan" con tratamiento "Cortar puntas" en esa franja. Entonces el sistema crea la cita con estado PENDING y hasOverlap = false, y aparece en el calendario.

Escenario 2: Creación con solapamiento detectado Dado que "Ruth Molina" ya tiene una cita de 10:00 a 11:00 el próximo lunes. Cuando se intenta crear otra cita para ese mismo empleado de 10:30 a 11:30. Entonces el sistema crea la cita pero marca hasOverlap = true y muestra advertencia visual en el calendario.

Escenario 3: Fallo por falta de tratamiento Dado que se intenta crear una cita sin seleccionar ningún tratamiento. Cuando se envía el formulario. Entonces el sistema devuelve error 400 y no crea la cita.

Escenario 4: Creación mediante wizard en el calendario Dado que el empleado hace clic en una franja horaria vacía del calendario. Cuando completa los 3 pasos del wizard (cliente+horario → tratamientos+empleado → confirmación). Entonces la cita se crea y aparece inmediatamente en el calendario sin recargar la página.

Anexo E — Pruebas de la API

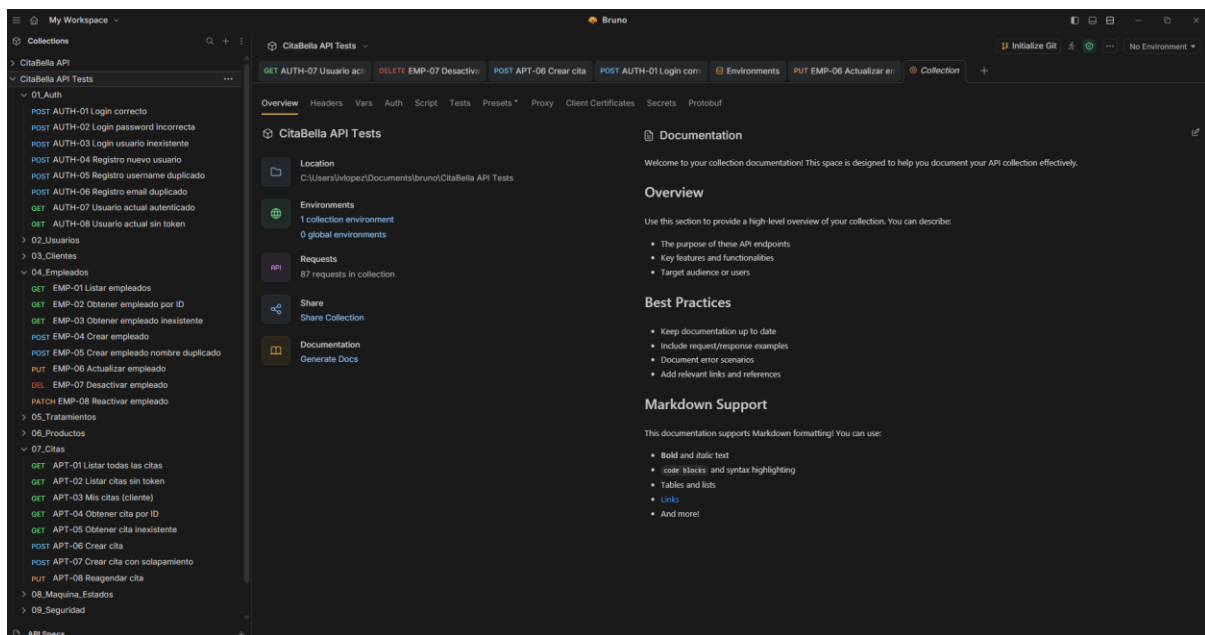


Ilustración 30 Bruno Battery Tests API Endpoints

ARCHIVO TESTS BRUNO EN:

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA/blob/main/docs/CitaBella-Bruno-Tests.zip

Anexo F — Prevención de riesgos laborales

ESTIMACIÓN DEL RIESGO		SEVERIDAD DE LAS CONSECUENCIAS		
		Ligeramente dañino	Dañino	Extremadamente dañino
PROBABILIDAD	Baja (B)	Riesgo trivial (T)	Riesgo tolerable (TO)	Riesgo moderado (MO)
	Media (M)	Riesgo tolerable (TO)	Riesgo moderado (MO)	Riesgo importante (I)
	Alta (A)	Riesgo moderado (MO)	Riesgo importante (I)	Riesgo intolerable (IN)

VALORACIÓN DEL RIESGO	ACCIÓN Y TEMPORALIZACIÓN
Trivial (T)	No se requiere acción específica.
Tolerable (TO)	No se necesita mejorar la acción preventiva. Sin embargo, se deben considerar soluciones más rentables o mejoras que no supongan una carga económica importante. Se requieren comprobaciones periódicas para asegurar que se mantiene la eficacia de las medidas de control.
Moderado (M)	Se deben hacer esfuerzos para reducir el riesgo, determinando las inversiones precisas. Las medidas para reducir el riesgo deben implantarse en un periodo determinado. Cuando el riesgo moderado esté asociado a consecuencias extremadamente dañinas, se precisará una acción posterior para establecer, con más precisión la probabilidad de daño; será la base para determinar la necesidad de mejorar las medidas de control.
Importante (I)	No debe comenzarse el trabajo hasta que se haya reducido el riesgo. Puede que se precisen recursos considerables para controlar el riesgo. Cuando se trate de un trabajo que se está realizando, debe remediarse el problema en un tiempo inferior al de los riesgos moderados.
Intolerable (IN)	No debe comenzar, ni continuar el trabajo hasta que se reduzca el riesgo. Si no es posible reducir el riesgo ni siquiera con recursos ilimitados, debe prohibirse el trabajo.

Ilustración 31 Prevención de riesgos Leyenda

Peligro identificado o puesto(s) afectado(s)	Severidad del daño			Probabilidad			Estimación del riesgo				
	Ligeramente dañino	Dañino	Extremadamente dañino	Baja	Media	Alta	Trivial	Tolerable	Moderado	Importante	Intolerable
SEGURIDAD Tropezones y caídas por cables u objetos en el suelo		X			X				X		
SEGURIDAD Incendios por cortocircuitos o uso inadecuado de equipos eléctricos			X	X					X		
AMBIENTAL Ruido excesivo por equipos informáticos		X			X				X		
AMBIENTAL Iluminación inadecuada			X			X			X		
ERGONOMICO Posturas forzadas y movimientos repetitivos (uso del ratón, teclado)		X				X				X	
ERGONOMICO Fatiga visual por exposición prolongada a pantallas		X				X				X	
PSICOSOCIAL Sobrecarga de trabajo, plazos ajustados, falta de reconocimiento		X			X				X	X	
PSICOSOCIAL Aislamiento social por trabajo individual			X			X			X		

Ilustración 32 Riesgos Laborales

	Sí	No	N/A	Observaciones
Consideraciones generales				
La utilización del equipo es segura, no es una fuente de riesgo por sí mismo.	☐	☐	☐	
Pantalla				
Los caracteres de la pantalla están bien definidos y tienen un tamaño suficiente.	☐	☐	☐	
El espacio entre caracteres y entre renglones es adecuado.	☐	☐	☐	
La imagen de la pantalla es estable y no se observan destellos, centelleos ni otras inestabilidades.	☐	☐	☐	
Se puede ajustar la luminosidad y el contraste entre los caracteres y el fondo de la pantalla.	☐	☐	☐	
La pantalla es orientable e inclinable.	☐	☐	☐	
No se observan reflejos ni reverberaciones molestas.	☐	☐	☐	
Teclado				
El teclado es inclinable e independiente de la pantalla.	☐	☐	☐	
Hay espacio suficiente delante del teclado para apoyar los brazos y las manos.	☐	☐	☐	
La superficie del teclado es mate y no presenta reflejos.	☐	☐	☐	
La disposición y las características de las teclas facilitan su utilización.	☐	☐	☐	
Los símbolos de las teclas resaltan y son fácilmente legibles.	☐	☐	☐	
Mesa o superficie de trabajo				
La superficie de la mesa es poco reflectante.	☐	☐	☐	
Las dimensiones son suficientes para colocar todos los elementos necesarios en el puesto de trabajo.	☐	☐	☐	
El portadocumentos es estable y regulable.	☐	☐	☐	
La ubicación del portadocumentos minimiza los movimientos incómodos de la cabeza y de los ojos.	☐	☐	☐	
El espacio de la superficie de trabajo es suficiente para permitir una posición cómoda.	☐	☐	☐	
Asiento de trabajo				
El asiento es estable, proporciona libertad de movimientos y permite adoptar una postura confortable.	☐	☐	☐	
La altura del asiento se puede regular.	☐	☐	☐	
El respaldo es reclinable y su altura ajustable.	☐	☐	☐	
Se pone un reposapiés a disposición de quien lo desee.	☐	☐	☐	

	Sí	No	N/A	Observaciones
Espacio				
El puesto de trabajo tiene dimensiones y espacio suficiente para permitir los cambios de postura y los movimientos de trabajo.	☐	☐	☐	
Iluminación				
Se garantiza un nivel adecuado de iluminación y unas relaciones adecuadas de luminancia entre la pantalla y su entorno.	☐	☐	☐	

Se evitan los deslumbramientos y los reflejos molestos mediante el acondicionamiento del puesto y la situación y las características de las fuentes de luz artificial.	☐	☐	☐	
Reflejos y deslumbramientos				
Los puestos de trabajo están instalados de manera que se evitan los reflejos molestos de las fuentes de luz natural y de los elementos claros del entorno.	☐	☐	☐	
Las ventanas están equipadas con algún dispositivo adecuado y regulable que atenúa la luz natural.	☐	☐	☐	
Ruido				
El ruido producido por los equipos instalados en el puesto de trabajo no perturba la atención ni la palabra.	☐	☐	☐	
Calor				
El calor emitido por los equipos instalados en el puesto de trabajo no ocasiona molestias.	☐	☐	☐	
Emisiones				
Las radiaciones electromagnéticas que no forman parte del espectro visible están reducidas a niveles insignificantes.	☐	☐	☐	
Humedad				
El nivel de humedad ambiental es aceptable.	☐	☐	☐	
Interconexión ordenador/persona				
El programa está adaptado a la tarea.	☐	☐	☐	
El programa es fácil de utilizar y se adapta a los conocimientos y a la experiencia de los usuarios.	☐	☐	☐	
Se informa a los trabajadores y se consulta con sus representantes sobre la existencia de posibles dispositivos de control.	☐	☐	☐	
El sistema (software) proporciona indicaciones sobre su desarrollo.	☐	☐	☐	
El sistema (software) muestra la información en un formato y a un ritmo adaptado a los trabajadores.	☐	☐	☐	
Se aplican los principios de la ergonomía al tratamiento de la información.	☐	☐	☐	

Gobierno de España. (s.f.). Prevención de riesgos laborales en la Unión Europea. Administración.gob.es. Consultado en febrero de 2026:

https://administracion.gob.es/pag_Home/Tu-espacio-europeo/derechos-obligaciones/ciudadanos/trabajo-jubilacion/seguridad-salud/prevencion-riesgos.html

Instituto Nacional de Seguridad y Salud en el Trabajo (INSST). (s.f.). Portal de prevención de riesgos laborales en el trabajo digital. Consultado en febrero de 2026:

<https://www.insst.es/>

Generalitat Valenciana. (s.f.). Procedimientos de trabajo. Portal de la Prevención de Riesgos Laborales de la Generalitat Valenciana. Consultado en febrero de 2026:

<https://prevencio.gva.es/es/fp-procedimientos-de-trabajo>

Jefatura del Estado. (1995). Ley 31/1995, de 8 de noviembre, de prevención de Riesgos Laborales. Boletín Oficial del Estado:

<https://www.boe.es/buscar/doc.php?id=BOE-A-1995-24292>

Anexo G — Instalación y despliegue del sistema

Requisitos previos

Software	Versión recomendada
Docker Desktop	Última estable
Docker Compose	v2
Git	Última estable
Navegador web moderno	Chrome, Edge o Firefox

Clonado del repositorio

git clone

https://github.com/pepolyente/2526_IVAN_LOPEZ_MOLINA_CITABELLA.git

cd 2526_IVAN_LOPEZ_MOLINA_CITABELLA

Configuración de variables de entorno

El proyecto incluye un fichero .env.example con la configuración base necesaria.

Crear el fichero .env:

cp .env.example .env

Modificar las variables necesarias:
MYSQL_DATABASE=citabella
MYSQL_USER=citabella
MYSQL_PASSWORD= citabella123
ADMIN_USERNAME=citabella
ADMIN_EMAIL=admin@citabella.com
ADMIN_PASSWORD=citabella123
JWT_SECRET=your_jwt_secret_citabella

Despliegue completo con Docker Compose

El entorno completo puede desplegarse mediante un único comando:

docker compose up -d --build

Anexo H — Estructura de documentación técnica

README.md del proyecto y la carpeta docs/

Documento	Contenido
Backend.md	Arquitectura backend y endpoints
Frontend.md	Estructura Angular
App-Android-IOS.md	Aplicación Flutter
DevOps.md	Docker, despliegue y variables
api-docs.html	Swagger UI estático
diagrams/	Diagramas UML y arquitectura
screenshots/	Capturas del sistema